EXP: 1 STUDY OF VARIOUS NETWORK COMMANDS USED

DATE: 16.07.2026 IN LINUX AND WINDOWS

WINDOWS

1. arp - address resolution protocol
2. hostname
3. ipconfig
4. nbtstat
5. netstat
6. nslookup
7. pathping
8. ping - packet internet gropher
9. route

Command Prompt - nslookup

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\TCS>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a               Displays current ARP entries by interrogating the current
                   protocol data.  If inet_addr is specified, the IP and Physical
                   addresses for only the specified computer are displayed.  If
                   more than one network interface uses ARP, entries for each ARP
                   table are displayed.
  -g               Same as -a.
  -v               Displays current ARP entries in verbose mode.  All invalid
                   entries and entries on the loop-back interface will be shown.
  inet_addr        Specifies an internet address.
  -N if_addr       Displays the ARP entries for the network interface specified
                   by if_addr.
  -d               Deletes the host specified by inet_addr. inet_addr may be
                   wildcarded with * to delete all hosts.
  -s               Adds the host and associates the Internet address inet_addr
                   with the Physical address eth_addr.  The Physical address is
                   given as 6 hexadecimal bytes separated by hyphens. The entry
                   is permanent.
  eth_addr         Specifies a physical address.
  if_addr          If present, this specifies the Internet address of the
                   interface whose address translation table should be modified.
                   If not present, the first applicable interface will be used.

Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.

C:\Users\TCS>hostname
DESKTOP-F4RMSER
```

Command Prompt - nslookup

```
C:\Users\TCS>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f               Clears the routing tables of all gateway entries.  If this is
                   used in conjunction with one of the commands, the tables are
                   cleared prior to running the command.

  -p               When used with the ADD command, makes a route persistent across
                   boots of the system. By default, routes are not preserved
                   when the system is restarted. Ignored for all other commands,
                   which always affect the appropriate persistent routes.

  -4               Force using IPv4.

  -6               Force using IPv6.

  command          One of these:
                     PRINT     Prints a route
                     ADD       Adds a route
                     DELETE    Deletes a route
                     CHANGE    Modifies an existing route
  destination      Specifies the host.
  MASK             Specifies that the next parameter is the 'netmask' value.
  netmask          Specifies a subnet mask value for this route entry.
                   If not specified, it defaults to 255.255.255.255.
  gateway          Specifies gateway.
  interface        the interface number for the specified route.
  METRIC           specifies the metric, ie. cost for the destination.
```

```
C:\Users\TCS>hostname
DESKTOP-I4RMSER

C:\Users\TCS>ipconfig

Windows IP configuration


Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::d51f:7aad:f478:5629%7
   IPv4 Address. . . . . . . . . . . : 172.16.11.192
   Subnet Mask . . . . . . . . . . . : 255.255.252.0
   Default Gateway . . . . . . . . . : 172.16.8.1

C:\Users\TCS>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

Command Prompt - nslookup

```
C:\Users\TCS>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    172.16.11.192:51810    4.213.25.240:https     ESTABLISHED
  TCP    172.16.11.192:51812    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51813    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51814    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51815    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51816    a23-211-60-142:https   CLOSE_WAIT
  TCP    172.16.11.192:51817    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51818    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51819    a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.11.192:51820    a23-199-67-57:https    CLOSE_WAIT
  TCP    172.16.11.192:51821    a23-199-67-57:https    CLOSE_WAIT
  TCP    172.16.11.192:52699    123:http               TIME_WAIT
  TCP    172.16.11.192:52700    123:http               TIME_WAIT
  TCP    172.16.11.192:52701    150.171.28.12:https    ESTABLISHED
  TCP    172.16.11.192:52702    a23-202-229-22:https   CLOSE_WAIT
  TCP    172.16.11.192:52703    4.207.44.66:https      TIME_WAIT
  TCP    172.16.11.192:52704    a23-202-229-22:https   ESTABLISHED
  TCP    172.16.11.192:52705    4.213.25.242:https     ESTABLISHED
  TCP    172.16.11.192:52706    123:http               ESTABLISHED

C:\Users\TCS>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.
```

```
C:\Users\TCS>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

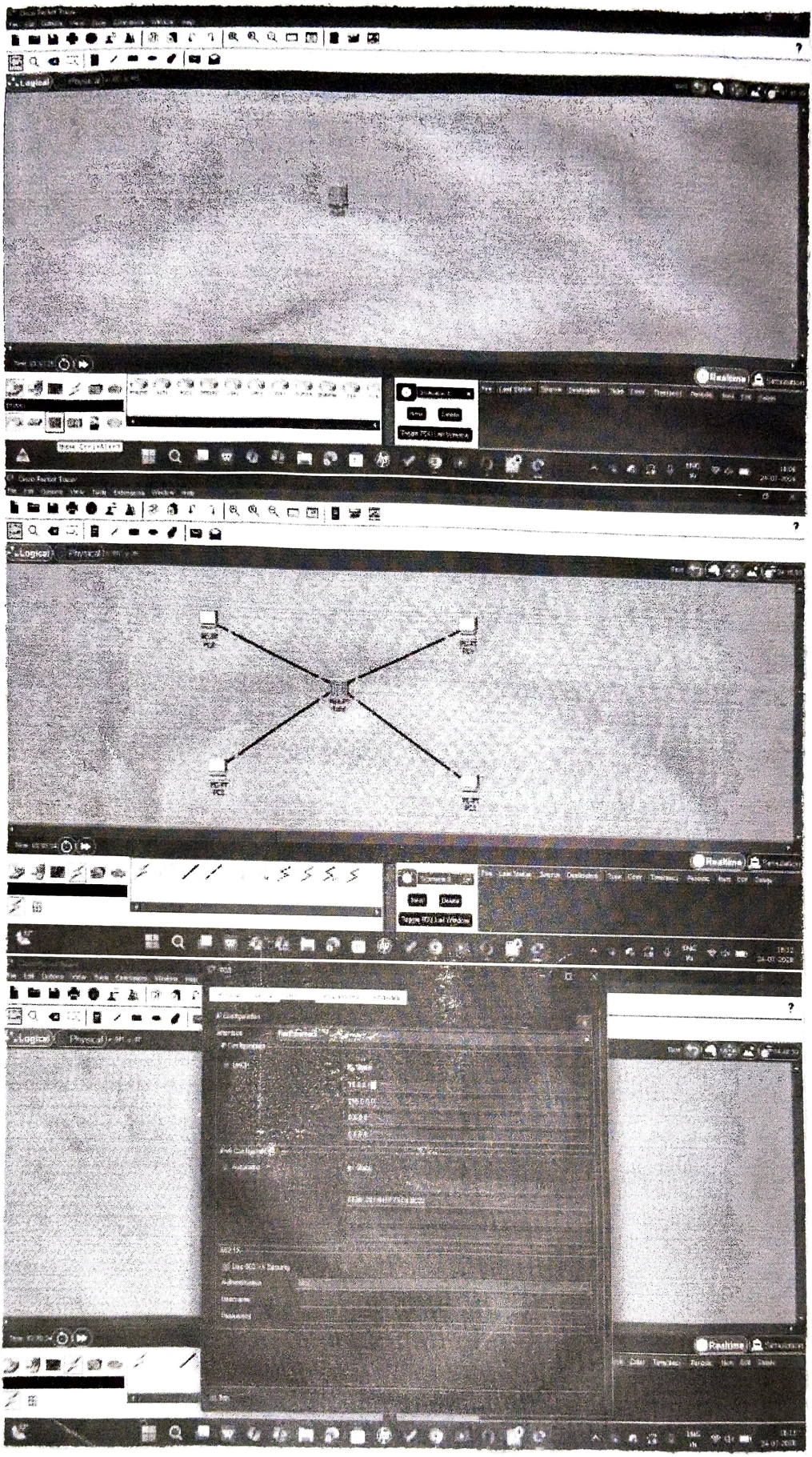
```
Command Prompt - nslookup

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.
Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*                .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
            destination^      ^mask      ^gateway     metric^    ^
                                                      Interface^
    If IF is not given, it tries to find the best interface for a given
    gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

    CHANGE is used to modify gateway and/or metric only.

    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32
```

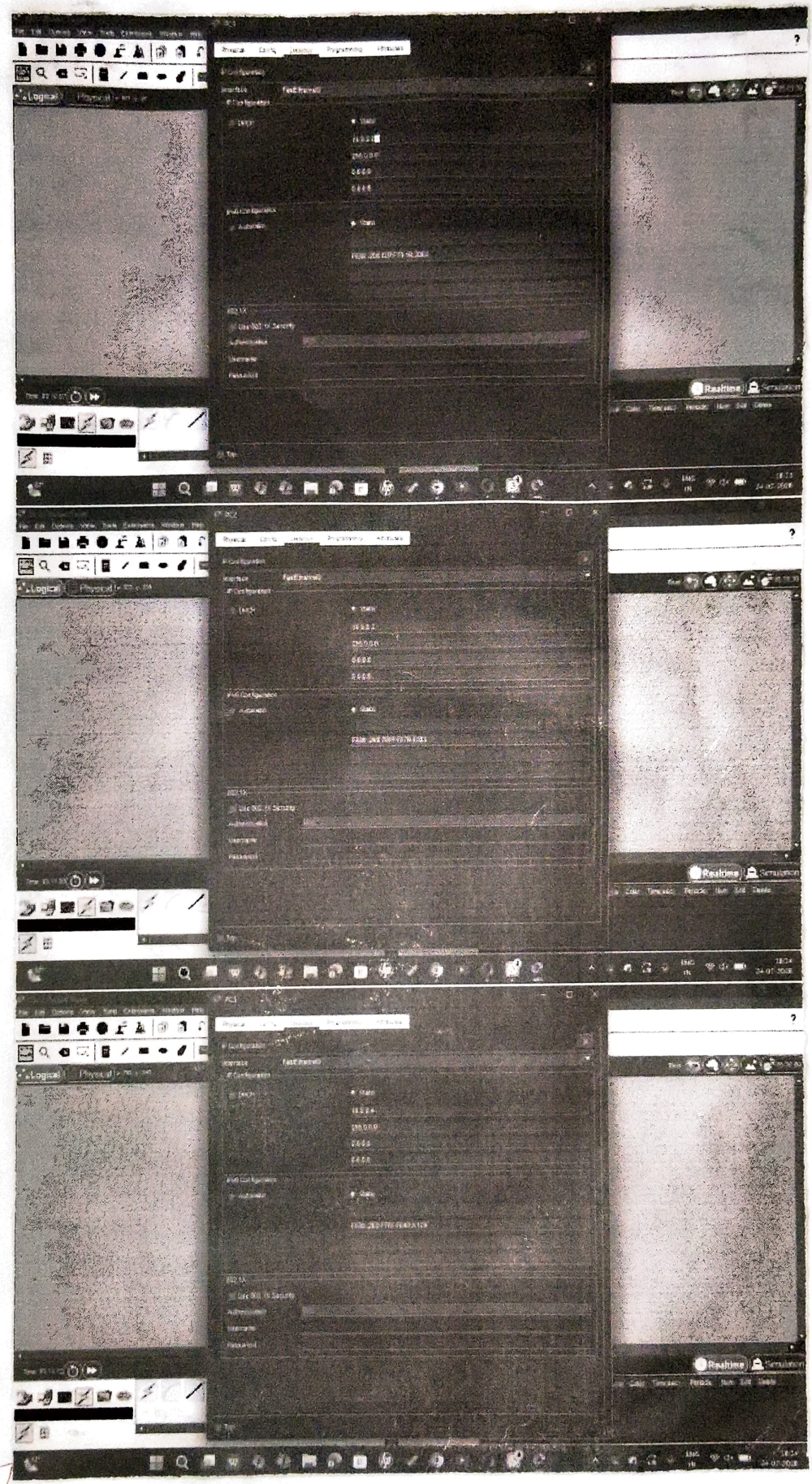
```
C:\Users\TCS>nslookup www.google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    www.google.com
Addresses:  2001:4860:4827:7700::
          2001:4860:482b:7700::
          2001:4860:4828:7700::
          2001:4860:4826:7700::
          2001:4860:482d:7700::
          2001:4860:4829:7700::
          2001:4860:482c:7700::
          2001:4860:482a:7700::
          142.251.156.119
          142.251.153.119
          142.251.157.119
          142.251.152.119
          142.251.151.119
          142.251.154.119
          142.251.155.119
          142.251.150.119

C:\Users\TCS>_
```

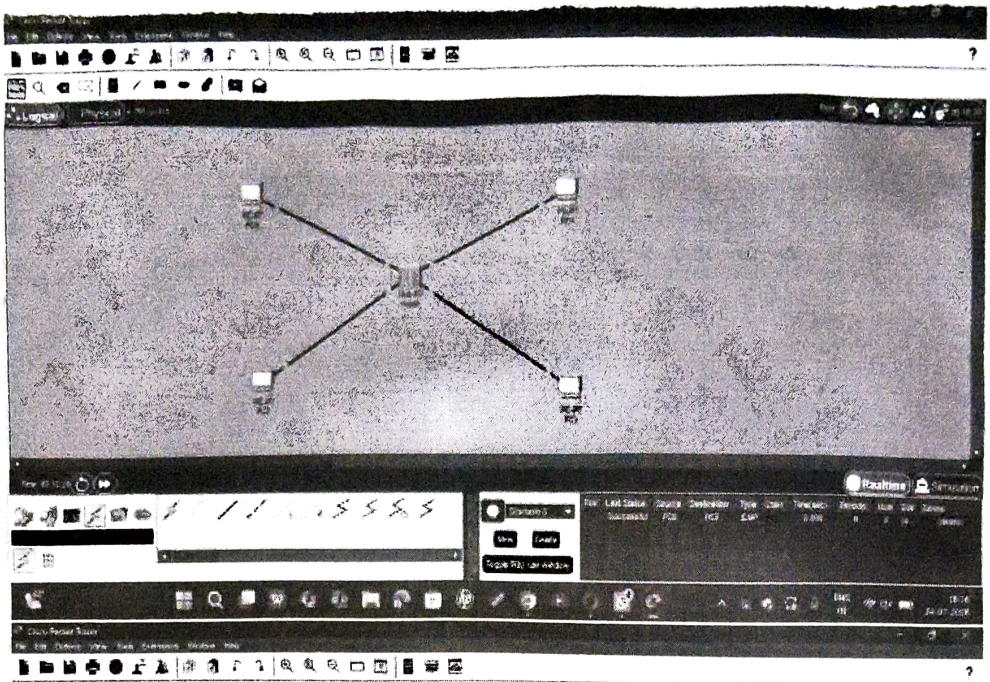
RESULT: Thus, various network commands used in Linux and windows have verified and executed successfu

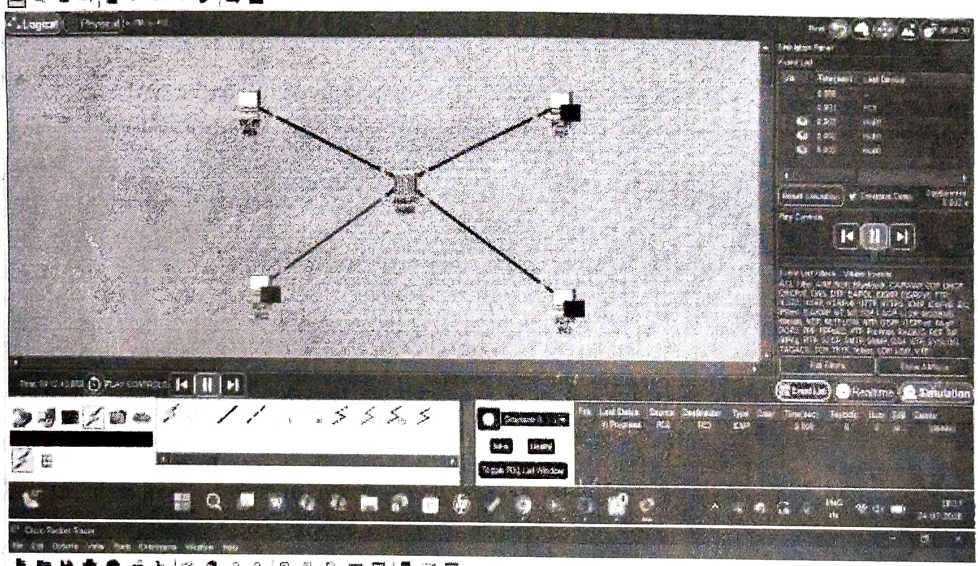
EXP. NO : 3
DATE: 22.07.2026

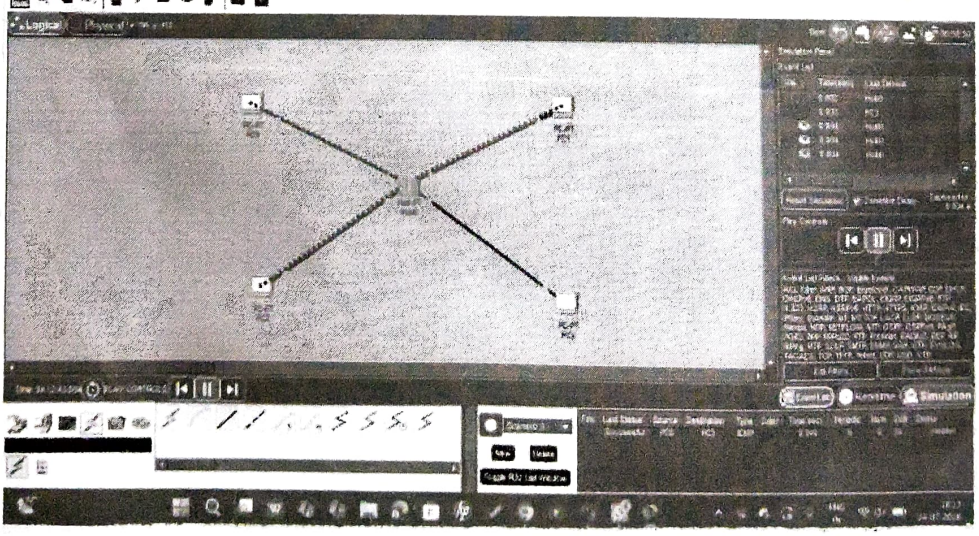
EXPERIMENT ON CISCO PACKET TRACKER (Simulation Tool)

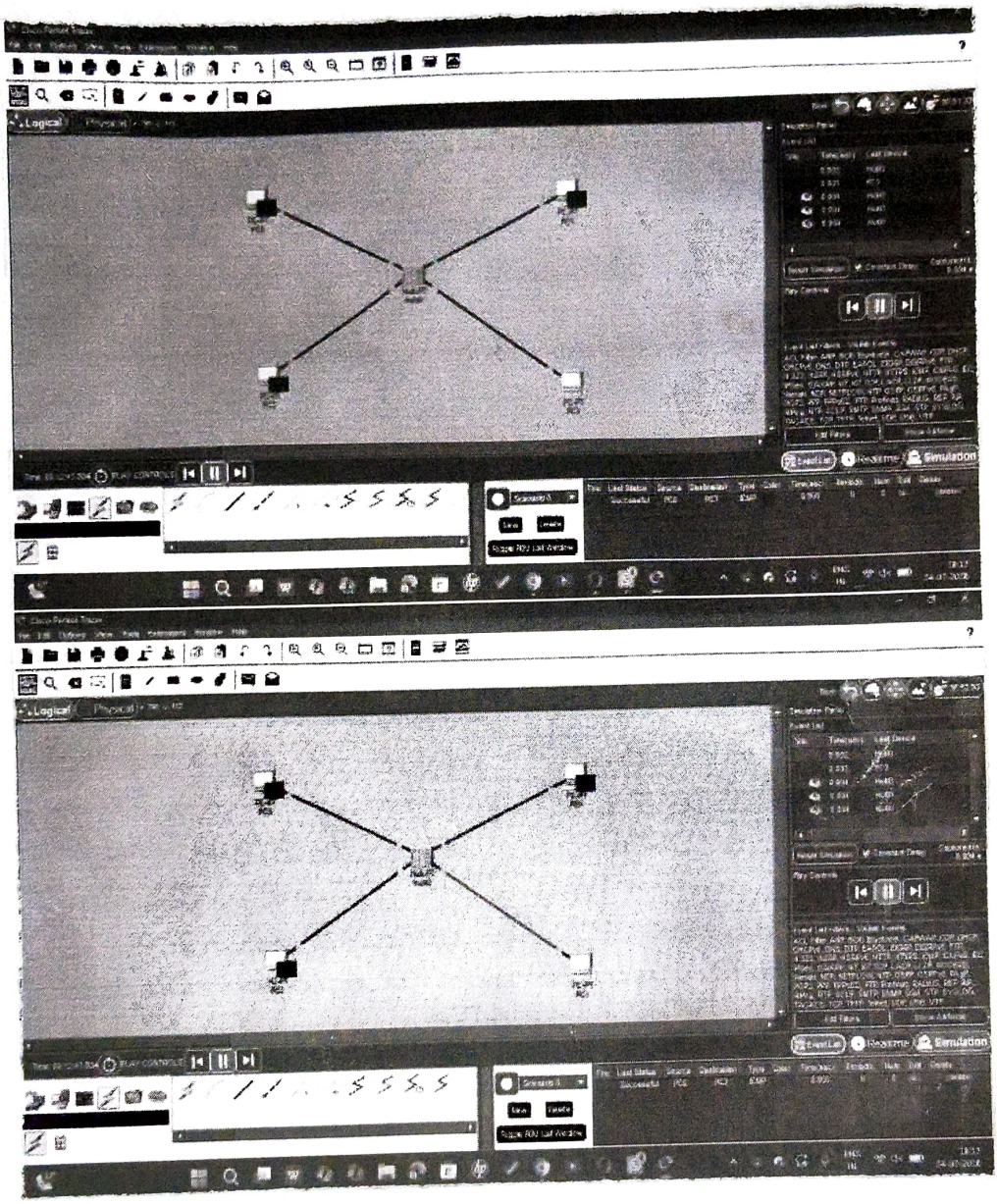












EXPLANATION:

1. Open Cisco Packet Tracer
2. Place one hub in the centre and four PCs in the workspace of Cisco Packet Tracer.
3. Connect each PC to the hub using Copper Straight-Through Cables.
4. Open each PC and navigate to Desktop → IP Configuration
5. Assign a unique IP address to each PC while keeping all the PCs in the same network.
6. Save the IP configuration and close the windows
7. Switch from Realtime mode to Simulation mode.
8. Use the Add Simple PDU tool to send a packet from one PC to another.
9. Click Capture / Forward or Auto Capture / Play to observe the movement of the packet.
10. Verify whether the packet reaches the destination PC successfully.

RESULT:

The network was successfully created using one hub and four PCs. Communication between the PCs was established successfully after assigning IP addresses, and the working of a hub was observed in Cisco Packet Tracer.

EXP. NO: 2

DATE: 22.07.2026 STUDY OF DIFFERENT TYPES OF NETWORK CABLES

a) UNDERSTAND DIFFERENT TYPES OF NETWORK CABLE:

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable.
2. Shielded Twisted Pair (STP) Cable.
3. Coaxial Cable.
4. Fibre Optic Cable.

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application / use
UTP	Category 3	10 bps	Advantages: → Cheaper in cost → Easy to install as they have a small overall diameter Disadvantages: → More prone to (EMI) Electromagnetic interference and noise.	10 Base - T Ethernet
	Category 5	Up to 100 Mbps		Fast Ethernet, Gigabit Ethernet
	Category 5e	1 Gbps		Fast Ethernet, Gigabit Ethernet
STP	category 6, 6a	10 Gbps	Advantages: → Shielded → Faster than UTP → Less susceptible to noise and interference Disadvantages: → Expensive → Greater installation effort.	Gigabit Ethernet, 10 G Ethernet, (55 m) Widely used in data centres.
	Category 7	10 Gbps		Gigabit Ethernet 10 G Ethernet (100 m)

a Coxial cable	RG-6 RG-59 RG-11	10 - 100 Mbps	• High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantages: • Limited distance • Cost • Size is bulky	Speed of signal is 500m Television network High speed internet connections.
fibre Optics cable	Single Mode Multi Mode	100 Gbps	Advantages: • High speed • High bandwidth • High security • Long distance Disadvantages • Expensive • Requires skilled installers	• Maximum distance of fibre optics cable is around 100 meters.

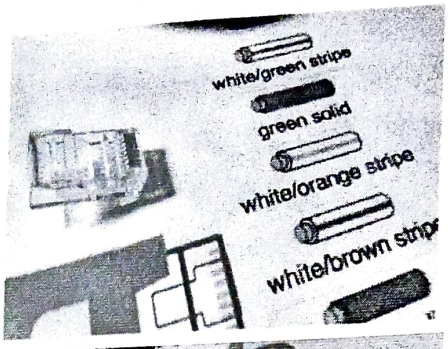
b) Make your own Ethernet Cross-over Cable / Straight Cable:

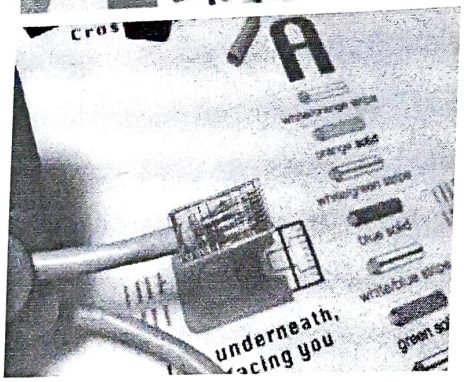
Tools and parts needed:

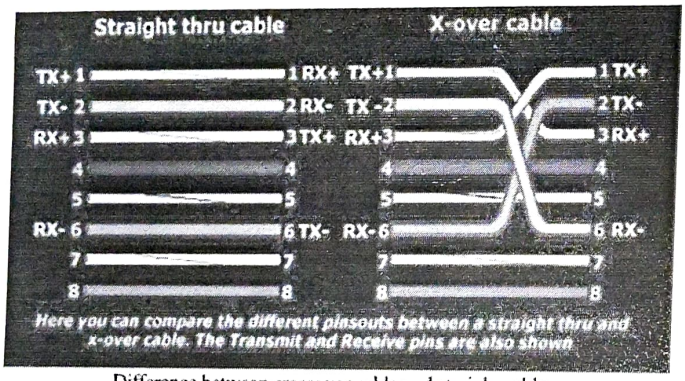
• Ethernet cabling, CAT5e is certified for gigabit support, but CAT5 cabling works as well, just over shorter distances.

• A crimping tool. This is an all-in one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables

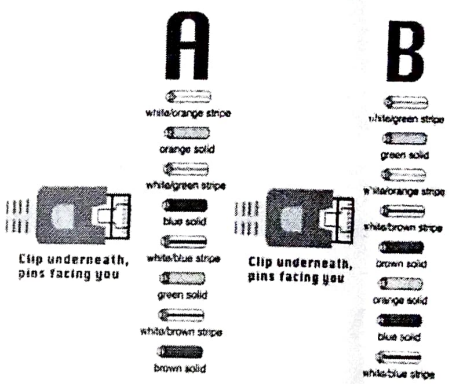
• Two RJ45 plugs.



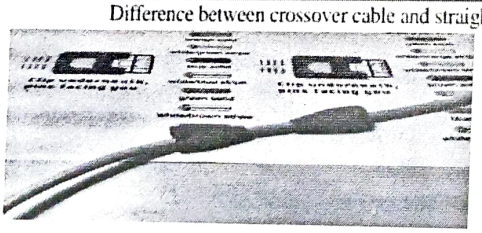


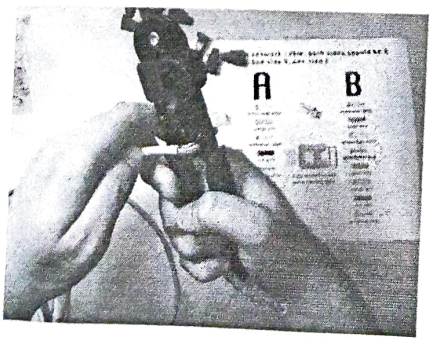


Straight through network cable: both sides should be A
Crossover cable: One side A, one side B



Difference between crossover cable and straight cable





- Optional two plug shields

STEP 1:

To start construction of device, begin by threading shields onto the cable.

STEP 2:

Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

STEP 3:

After, you will need to untangle the wires; there should be four "twisted pairs." Referring back to the sheet, arrange them from top to bottom. One end should be in arrangement A and other in B.

STEP 4:

Arrange the wires in the correct order, trim them evenly and insert them into the RJ45 plug with the clip facing away and the gold pins facing you.

STEP 5:

Push the cable fully in the plug so the cable jacket is inside the connector. Trim and reinsert if needed.

STEP 6:

Place the RJ45 plug into the trimming tool and trim it firmly without applying excessive force.

STEP 7:

Repeat for the other end using B for a crossover cable or A for a straight-through cable. Test the cable by connecting two devices

RESULT:

Thus, different types of network cables have been studied successfully.

EXP. NO: 4
DATE: 30.07.26

SETTING AND CONFIGURING A LOCAL AREA NETWORK

AIM:

Setup and configure a LAN (Local area network) using a switch and Ethernet cables in your lab.

What is a LAN?

A Local area network (LAN) refers to a network that connects devices within a limited area, such as an office building, school or home. It enables users to share resources, including data and internet access. LAN connects devices to promote collaboration and transfer information between users such as computers printers and switches. A local area network switch serves as primary connecting device, managing and directing communications within local network. Each connected device on LAN switch can communicate directly with each other, allowing fast and secure data access.

How to set up a LAN?

STEP 1:

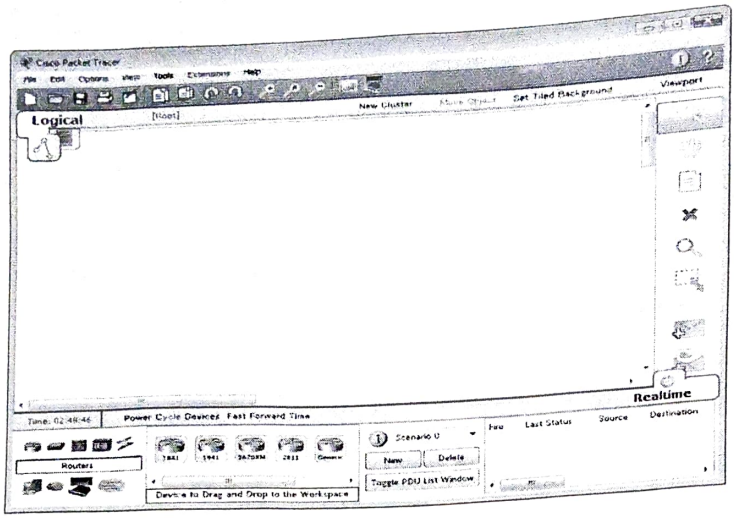
Plan and Design an appropriate network topology taking into account network requirements and equipment location.

STEP 2:

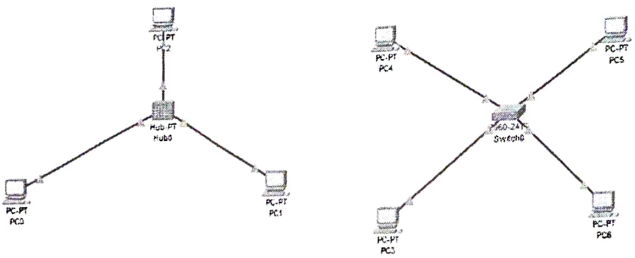
You can take 4 computers, a switch with 8, 16 or 24 ports which is sufficient for networks of these sizes and 4 Ethernet cables.

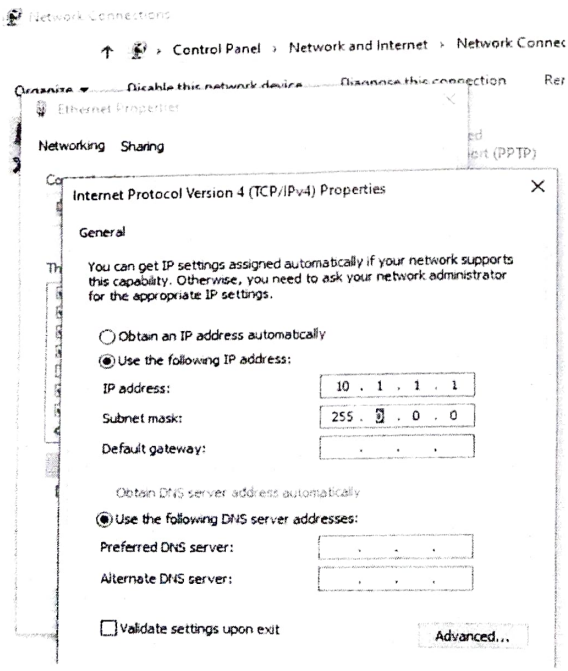
STEP 3:

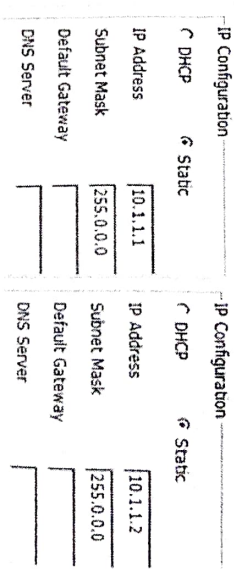
Connect your computers to network switch via an Ethernet cable, which is as simple as plugging one end of the Ethernet cable into your computer and


Cisco Packet Tracer
Logical
Realtime
Power Cycle Devices
Fast Forward Time
Routers
Devices to Drag and Drop to the Workspace
New
Delete
Toggle PDU List Window

File Edit Options View Tools Extensions Window Help
Logical
Physical


PC-PT PC2
Hub-PT Hub0
PC-PT PC0
PC-PT PC1
PC-PT PC4
PC-PT PC5
2960-24TT Switch0
PC-PT PC3
PC-PT PC6


Network Connections
Control Panel › Network and Internet › Network Connec
Ethernet Properties
Networking Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...


IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.1
255.0.0.0
IP Configuration
DHCP
Static
IP Address
Subnet Mask
Default Gateway
DNS Server
10.1.1.2
255.0.0.0

IP Configuration
IP Configuration

other end into your network switch.

STEP 4: Assign IP address to your PCs.

1. Log on to the client computer as Administrator or as Owner.

2. Click Network and Internet connections.

3. Right Click Local Area Connection / Ethernet → Go to Properties → Select Internet Protocol (TCP/IPV4) → Click on properties → Select use the following ip address option and assign ip address.

4. Similarly assign IP address to all the PCs connected to switch.

Eg: PC1 - IP address : 10.1.1.1, subnet mask 255.0.0.0
PC2 - IP address : 10.1.1.2, subnet mask 255.0.0.0

STEP 5: Configure a network switch

1. Connect your computer to the switch: To access the switch's web interface, you will need to connect your computer to switch using an Ethernet cable.

2. Log in to the web interface: Open a web browser and enter the IP address of the switch in the address bar.

3. Configure basic settings: Once you're logged in, you will be able to configure basic settings for switch.

4. Assign IP address as: 10.1.1.3, subnet mask 255.0.0.0.

STEP 6:

Check the connectivity between switch and other machine by using ping command in the command prompt of the device.

STEP 7:

Select a folder → Go to properties → Click Sharing tab → share it with everyone on the same LAN.

STEP 8:

Try to access the shared folder from others computers of the network.

RESULT:

A LAN was successfully setup using a switch and Ethernet cables. IP addresses were assigned to all computers and connectivity was verified using ping command and a folder was successfully shared and accessed between computers on same network.

EXP.NO: 5

DATE: 06.08.2026 PACKET CAPTURE USING WIRESHARK

AIM:

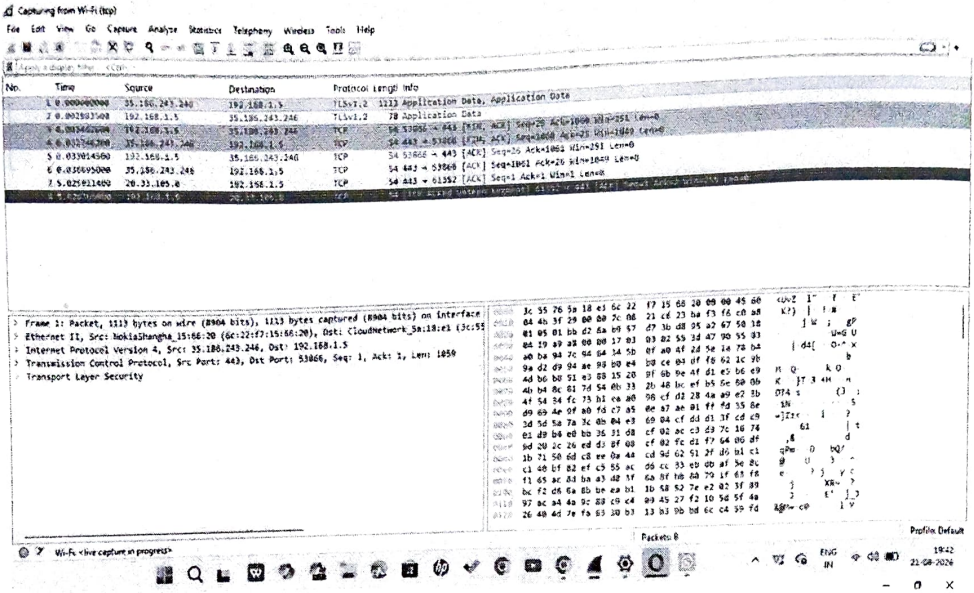
To capture filter inspect and analyse network packets using the wireshark packet analyzer

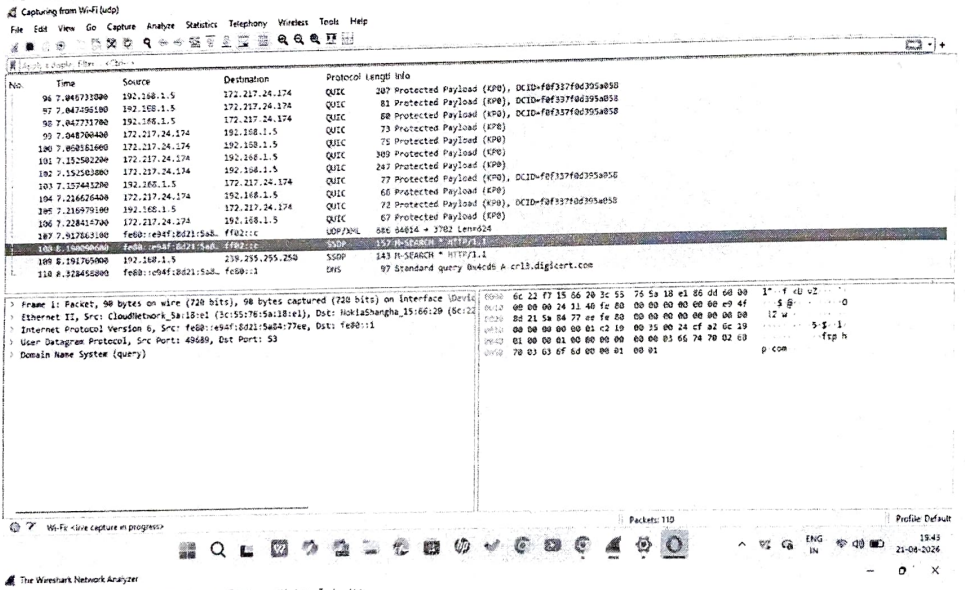
ALGORITHM:

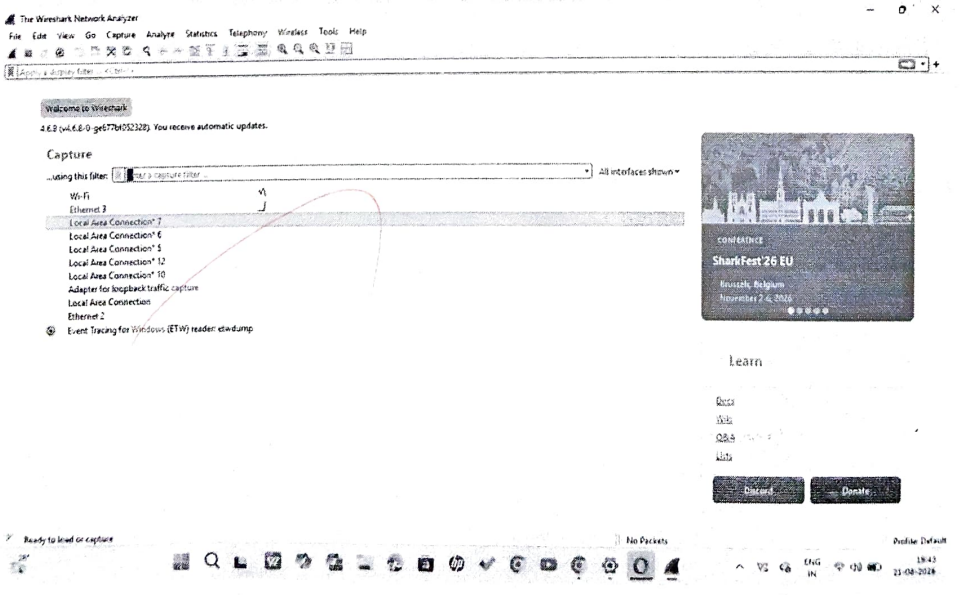
1. Open wireshark
2. Select the active network interface
3. Configure capture to stop after 100 packets
4. Start Packet Capture
5. Apply display filters
6. Inspect packed details and view the flow graph.
7. Save the captured packets.

PROCEDURE:

- Launch wireshark and Subset the active network interface
- Go to capture → options and set stop capture after 100 packets.
- Click start to capture packet
    1. TCP / UDP : tcp || udp
    2. ARP : arp
    3. DNS : dns
    4. Http : http
- Save the capture file to future analysis.


Capturing from Wi-Fi (tcp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
1 0.000000000 35.186.243.246 192.168.1.5 TLSv1.2 1213 Application Data, Application Data
2 0.002933000 192.168.1.5 35.186.243.246 TLSv1.2 78 Application Data
3 0.003462000 192.168.1.5 35.186.243.246 TCP 66 53866 → 443 [FIN, ACK] Seq=26 Ack=1060 Win=251 Len=0
4 0.032746000 35.186.243.246 192.168.1.5 TCP 54 443 → 53866 [FIN, ACK] Seq=1060 Ack=25 Win=1049 Len=0
5 0.033014000 192.168.1.5 35.186.243.246 TCP 54 53866 → 443 [ACK] Seq=26 Ack=1061 Win=251 Len=0
6 0.036695000 35.186.243.246 192.168.1.5 TCP 54 443 → 53868 [ACK] Seq=1061 Ack=26 Win=1049 Len=0
7 5.025811000 20.33.105.0 192.168.1.5 TCP 54 443 → 61352 [ACK] Seq=1 Ack=1 Win=1 Len=0
8 5.025934000 192.168.1.5 20.33.105.0 TCP 54 [TCP Keep-Alive] 61352 → 443 [ACK] Seq=0 Ack=1 Win=251 Len=0
Frame 1: Packet, 1213 bytes on wire (8904 bits), 1213 bytes captured (8904 bits) on interface
Ethernet II, Src: HokiaShangha_15:66:20 (6c:22:f7:15:66:20), Dst: CloudNetwork_5a:18:e1 (3c:55
Internet Protocol Version 4, Src: 35.186.243.246, Dst: 192.168.1.5
Transmission Control Protocol, Src Port: 443, Dst Port: 53866, Seq: 1, Ack: 1, Len: 1059
Transport Layer Security
Wi-Fi: <live capture in progress>
Packets: 8
Profile: Default
ENG IN
19:42
21-08-2026


Capturing from Wi-Fi (udp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
96 7.045731000 192.168.1.5 172.217.24.174 QUIC 207 Protected Payload (KP0), DCID=f8f337f0d395a058
97 7.047598100 192.168.1.5 172.217.24.174 QUIC 81 Protected Payload (KP0), DCID=f8f337f0d395a058
98 7.047731700 192.168.1.5 172.217.24.174 QUIC 80 Protected Payload (KP0), DCID=f8f337f0d395a058
99 7.048700400 172.217.24.174 192.168.1.5 QUIC 73 Protected Payload (KP0)
100 7.050161600 172.217.24.174 192.168.1.5 QUIC 75 Protected Payload (KP0)
101 7.152502200 172.217.24.174 192.168.1.5 QUIC 309 Protected Payload (KP0)
102 7.152503800 172.217.24.174 192.168.1.5 QUIC 247 Protected Payload (KP0)
103 7.157443200 192.168.1.5 172.217.24.174 QUIC 77 Protected Payload (KP0), DCID=f8f337f0d395a058
104 7.216626400 172.217.24.174 192.168.1.5 QUIC 66 Protected Payload (KP0)
105 7.216979100 192.168.1.5 172.217.24.174 QUIC 72 Protected Payload (KP0), DCID=f8f337f0d395a058
106 7.228413700 172.217.24.174 192.168.1.5 QUIC 67 Protected Payload (KP0)
107 7.917843100 fe80::e94f:8d21:5a8.. ff02::c UDP/XML 886 64814 → 3702 Len=824
108 8.190696600 fe80::e94f:8d21:5a8.. ff02::c SSDP 157 M-SEARCH * HTTP/1.1
109 8.191765000 192.168.1.5 239.255.255.250 SSDP 143 M-SEARCH * HTTP/1.1
110 8.328455800 fe80::e94f:8d21:5a8.. fe80::1 DNS 97 Standard query 0x4cd8 A crl3.digicert.com
Frame 1: Packet, 98 bytes on wire (728 bits), 98 bytes captured (728 bits) on interface \Devic
Ethernet II, Src: CloudNetwork_5a:18:e1 (3c:55:76:5a:18:e1), Dst: HokiaShangha_15:66:20 (6c:22
Internet Protocol Version 6, Src: fe80::e94f:8d21:5a24:77ee, Dst: fe80::1
User Datagram Protocol, Src Port: 49680, Dst Port: 53
Domain Name System (query)
Wi-Fi: <live capture in progress>
Packets: 110
Profile: Default
ENG IN
19:43
21-08-2026


The Wireshark Network Analyzer
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Welcome to Wireshark
4.6.3 (v4.6.3-0-ge677b4052328). You receive automatic updates.
Capture
...using this filter:
All interfaces shown
Wi-Fi
Ethernet 3
Local Area Connection* 7
Local Area Connection* 6
Local Area Connection* 5
Local Area Connection* 12
Local Area Connection* 10
Adapter for loopback traffic capture
Local Area Connection
Ethernet 2
Event Tracing for Windows (ETW) reader: etwdump
CONFERENCE
SharkFest'26 EU
Brussels, Belgium
November 2-6, 2026
Learn
Docs
FAQ
Q&A
Lists
Discord
Donate
Ready to load or capture
No Packets
Profile: Default
ENG IN
19:43
21-08-2026

RESULT:

Successfully captured 100 network packets, applied display filter to TCP, UDP analyzed packet details using wireshark.

EXP. NO: 6

DATE: 13.8.26 IMPLEMENTATION OF ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection and correction using Hamming code concept. Make a test run to input data stream and verify error correction feature.

ALGORITHM:

1. Start
2. Read the text data from sender
3. Convert each character of text into 8-bit Ascii binary form.
4. Calculate the no. of redundant bits using Hamming code.
5. Insert redundant bits at positions 1, 2, 4, 8....
6. Calculate the parity bits and generate the Hamming code.
7. Store the generated code in channel file.
8. At the receiver, read the data from the channel file.
9. Calculate the parity bits again to find error position.
10. If the error position is non-zero, correct the bit at that position.
11. Remove the redundant bits.
12. Convert the remaining binary data into Ascii characters.
13. Display the corrected message.
14. Stop.

CODE:

```
def hamming(d):
    m = len(d); r=0
    while 2**r < m+r+1: r+=1
    a = ['0'] * (m+r); j=0
    for i in range(1, len(a)+1)
        if i & (i-1): a[i-1] = d[j]; j+=1
    for p in [1<<i for i in range(r)]:
        x=0
        for i in range(1, len(a)+1):
            if i & p: x^=int(a[i-1])
        a[p-1] = str(x)
    return ''.join(a)

s = input("Enter data: ")
b = ''.join(format(ord(c), '08b') for c in s)
c = hamming(b)
open("channel", "w").write(c)
print("Binary Data: ", b)
print("Channel Data:", c)
c = open("channel").read()
e = int(input("Enter error position (0 for no error): "))
if e: c = c[:e-1] + str(1-int(c[e-1])) + c[e:]
r=0
while 2**r < len(c)+1: r+=1
x=0
for p in [1<<i for i in range(r)]:
    x=0
    for i in range(1, len(c)+1):
        if i & p: x^=int(c[i-1])
    if x: x+=p
print("Error position:", x)
if x: c = c[:x-1] + str(1-int(c[x-1])) + c[x:]
d = ''.join(c[i] for i in range(len(c)) if (i+1)&i)
print("Received Data:", ''.join(chr(int(d[i:i+8],2)) for i in
                                range(0, len(d), 8)))
```

OUTPUT:

```
Enter data : A
Binary Data: 01000001
Channel Data: 11010000011
Enter error position (0 for no error): 5
Error position : 5
Received Data : A
```

RESULT:

Hamming code successfully detects the error position and corrects the erroneous bit, recovering the original data.

EXP. NO: 7
DATE: 20.8.26

FLOW CONTROL AT THE DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

AIM:

To implement Flow control at the Data Link Layer using the Sliding Window Protocol and simulate the transmission of frames from one node to another.

ALGORITHM:

1. Start the program
2. Set the total number of frames and size
3. Sender sends frames equal to the current window size.
4. Receiver receives the frames and sends acknowledgements
5. After receiving Acks, the sender slides the window forward
6. Continue until all frames are transmitted.
7. Display the frame transmission and acknowledgement process.
8. Stop

CODE:

```
def sliding-window (total-frames, window-size):
    sender = 0
    print ("Sliding window Protocol")
    print ("----------")
    while sender < total-frames:
        end = min (sender + window-size, total-frames)
        print ("\n Sender sends frames:")
        for frame in range (sender, end):
            print ("Frame", frame, "sent")
        print ("Receiver sends acknowledgements:")
        for frame in range (sender, end):
            print ("Ack", frame, "received")
```

```
    sender = end
    if sender < total_frames:
        print ("Window slides forward...")
print ("In All frames transmitted successfully.")
total_frames = int(input("Enter total number of frames:"))
window_size = int(input("Enter window size:"))
sliding_window(total_frames, window_size)
```

OUTPUT:

```
Enter total number of frames: 8
Enter window size: 3
Sliding Window Protocol
----------------
Sender sends frames:
Frame 0 sent
Frame 1 sent
Frame 2 sent
Receiver sends acknowledgements:
Ack 0 received
Ack 1 received
Ack 2 received
Window slides forward...
Sender sends frames:
Frame 6 sent
Frame 7 sent
Receiver sends acknowledgements:
Ack 6 received
Ack 7 received
All frames transmitted successfully
```

RESULT:

The Sliding Window protocol was successfully implemented in Python, simulating frame transmission and acknowledgements using a fixed-size window.

EXP.NO: 8 a)
DATE:

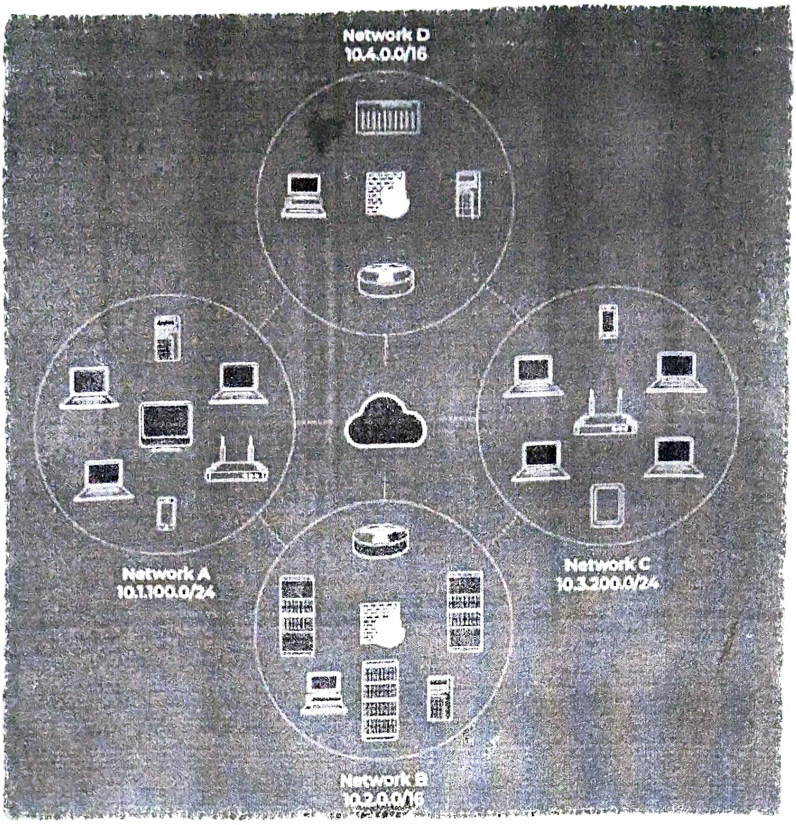
Nmap Live Host Discovery using ARP, ICMP, TCP and UDP

AIM:

To discover live hosts using Nmap scans such as ARP, ICMP, TCP and UDP on the TryHackMe platform.

ALGORITHM:

1. Open the TryHackMe platform and access the Nmap Live Host Discovery room.
2. Study the concept of live host discovery before performing network scanning.
3. Start with an ARP scan to identify live hosts in the local network.
4. Observe the ARP requests and responses from the target hosts.
5. Perform an ICMP scan to check whether the target systems respond to ICMP requests.
6. Perform an TCP SYN ping scan by sending TCP packets to the required ports.
7. Perform an TCP Ack ping scan and observe the response from the target host.
8. Perform a UDP ping scan to determine whether the target system is reachable.
9. Compare the results obtained from ARP, ICMP, TCP and UDP host-discovery methods.
10. Record the live host identified by Nmap and study the scan results.



Send Packet

From:

computer1

To:

computer1

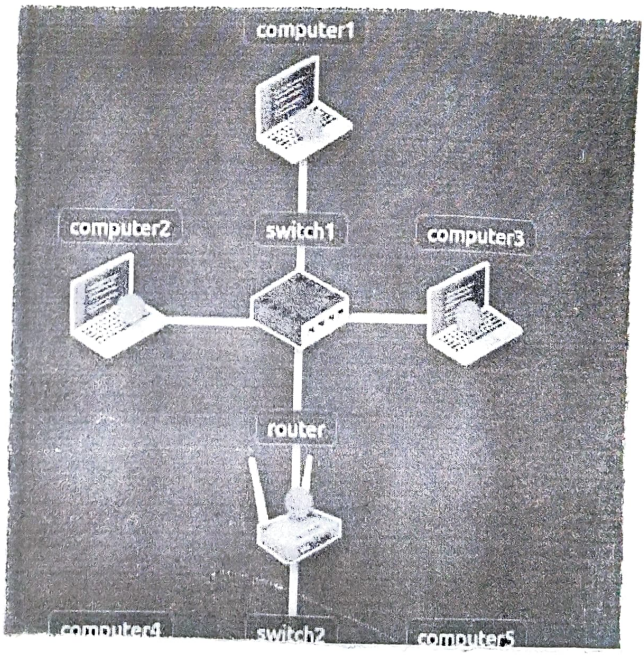
Packet Type:

arp_request

Data:

computer6

Send Packet



Send Packet

From:

computer4

To:

computer4

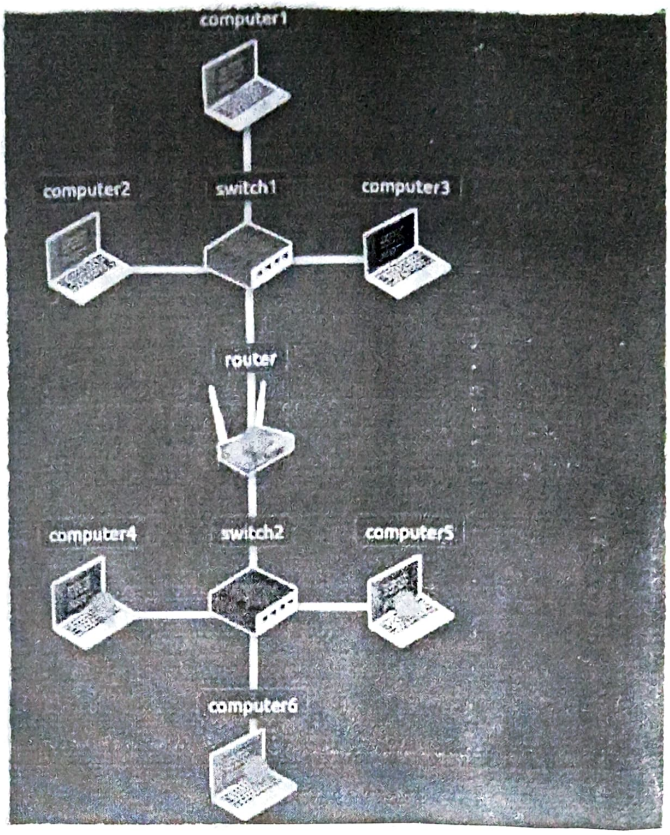
Packet Type:

arp_request

Data:

computer6

Send Packet



	ISO/OSI	TCP/IP	
7	Application Layer	Application Layer	HTTP, HTTPS, SMTP, POP3, IMAP, SSH, FTP, SNMP, Telnet, RDP...
6	Presentation Layer		
5	Session Layer		
4	Transport Layer	Transport Layer	TCP, UDP
3	Network Layer	Network Layer	IPv4, IPv6, ICMP, IPsec
2	Data Link Layer	Link Layer	ARP, Ethernet (802.3), WiFi (802.11), DSL, Bluetooth
1	Physical Layer		

EXP NO: 9

DATE:

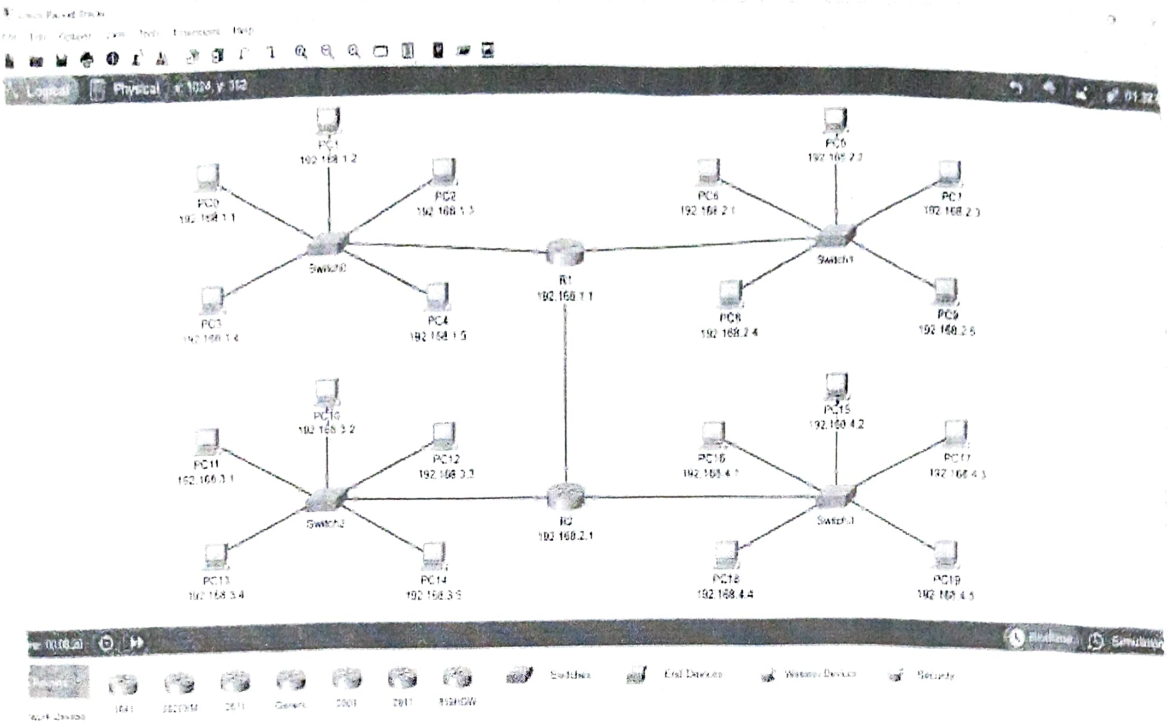
SUBNETTING

AIM:

To design and configure a subnetted network using Cisco Packet Tracer.

ALGORITHM:

1. Open Cisco Packet Tracer
2. Add a router, switch and PCs
3. Connect the devices using suitable cables.
4. Open the router CLI
5. Configure the router interfaces with IP addresses and subnet masks.
6. Enable the interfaces using no shutdown.
7. Configure the IP address and subnet mask of each PC.
8. Set the appropriate default gateway for each PC.
9. Use the ping command to test connectivity.
10. Verify communication between the different subnets



RESULT:

Thus, the subnetted network was successfully configured and connectivity was verified using ping.

EXP. NO: 10 a.

DATE:

INTERNETWORKING WITH ROUTER

AIM:

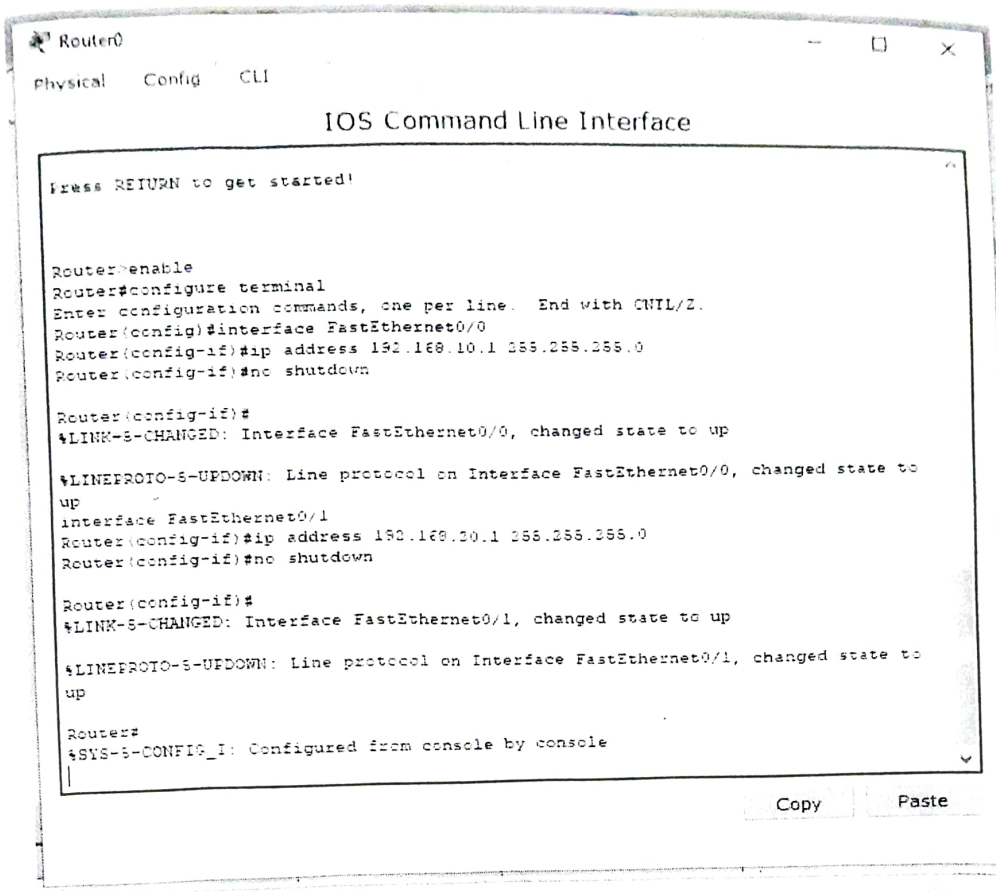
To design and configure an internetwork using a router in Cisco Packet Tracer.

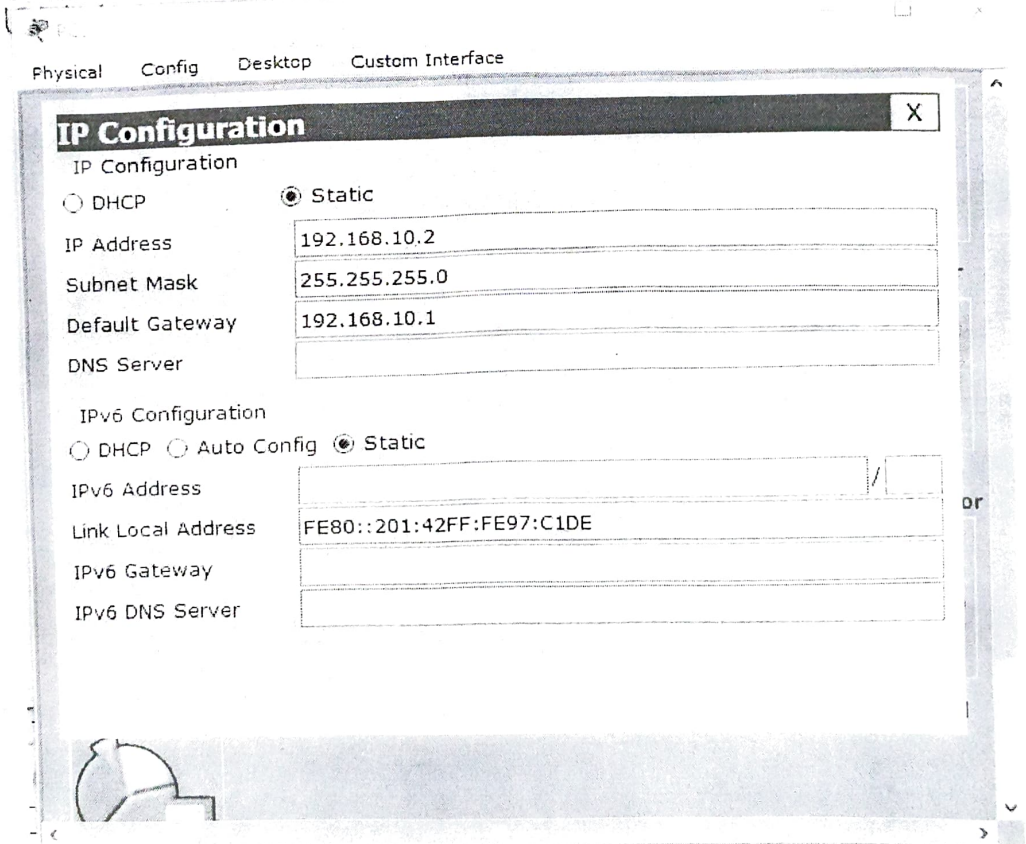
ALGORITHM:

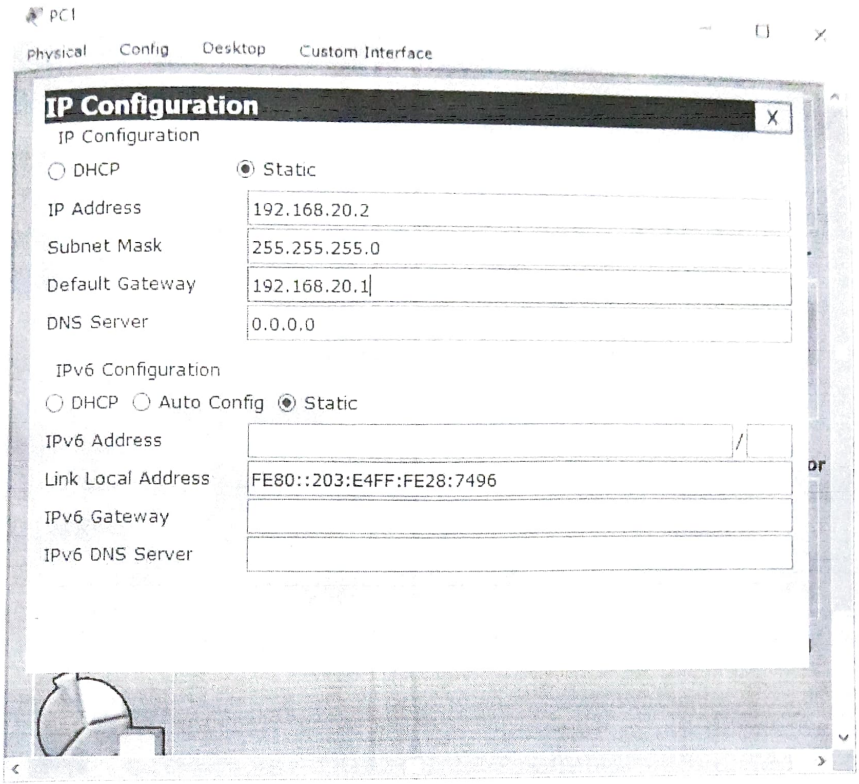
1. Open Cisco Packet Tracer.
2. Add one router and two PCs.
3. Connect the PCs to the router.
4. Configure the router interfaces with IP addresses.
5. Enable the interfaces using no shutdown.
6. Configure IP address and subnet mask for PC0.
7. Configure IP address and subnet mask for PC1
8. Set the default gateway for both PCs.
9. Send a PDU from PC0 to PC1.
10. Verify the communication between the two networks.

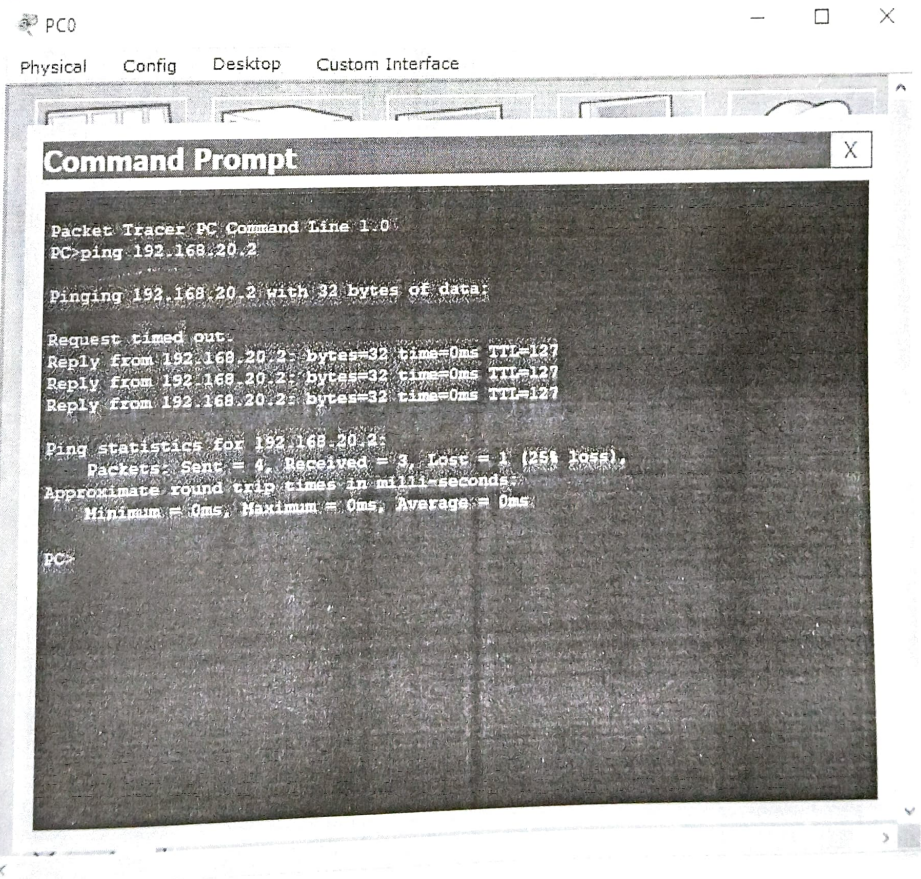
RESULT:

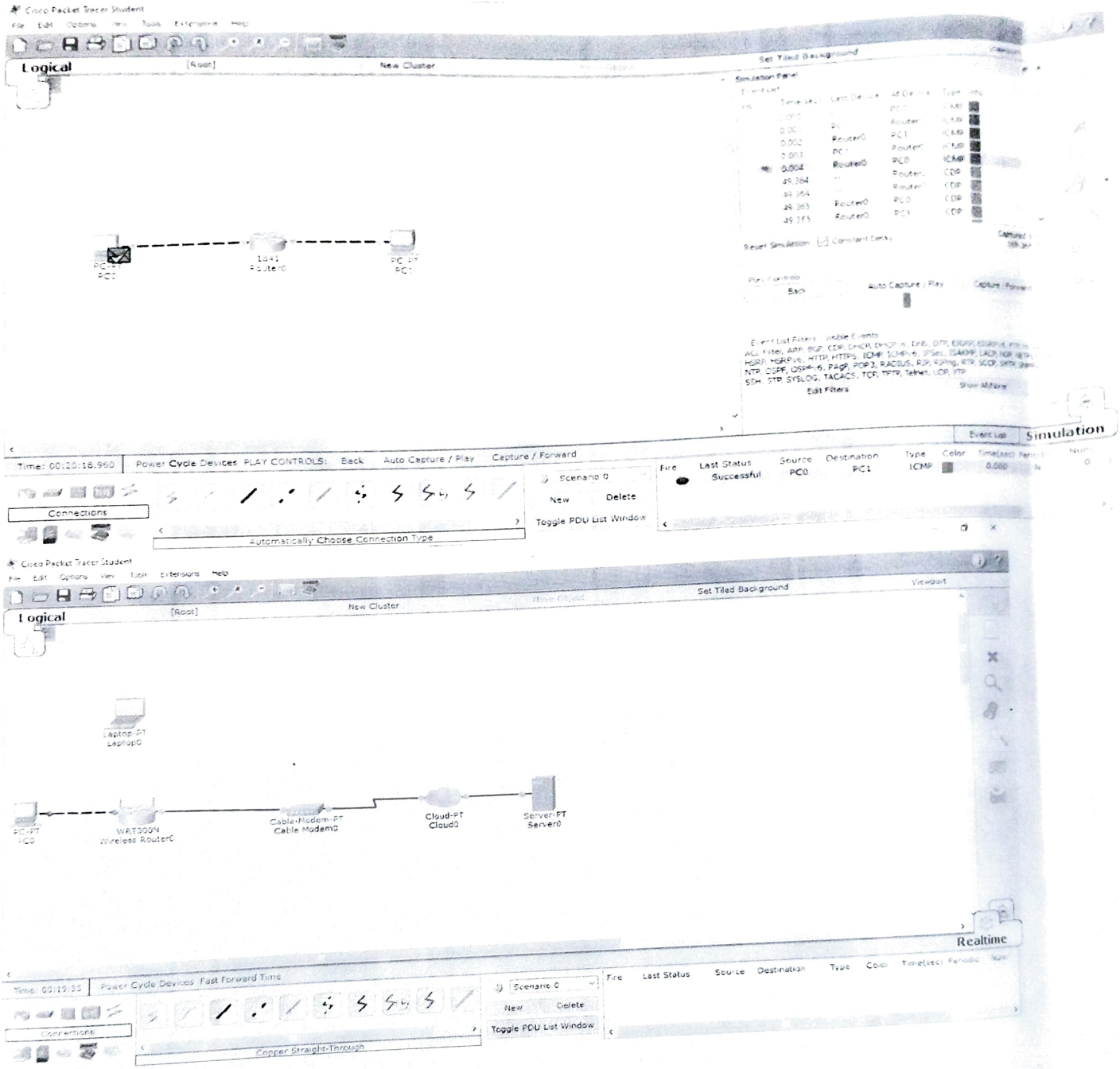
Thus, the internetwork was successfully configured using a router and connectivity was verified.

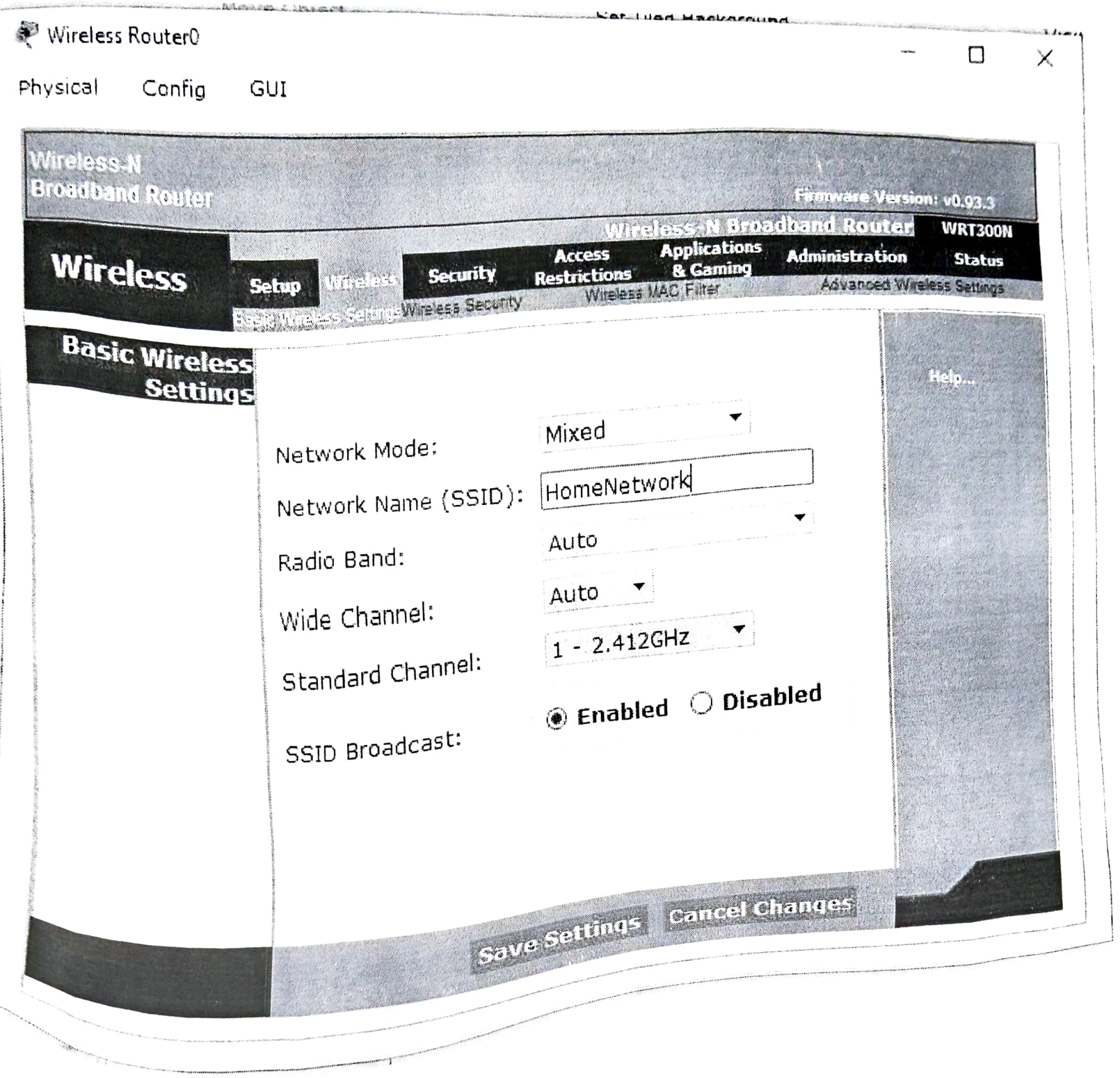

Router0
Physical
Config
CLI
IOS Command Line Interface
Press RETURN to get started!
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface FastEthernet0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
interface FastEthernet0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
Router#
%SYS-5-CONFIG_I: Configured from console by console
Copy
Paste

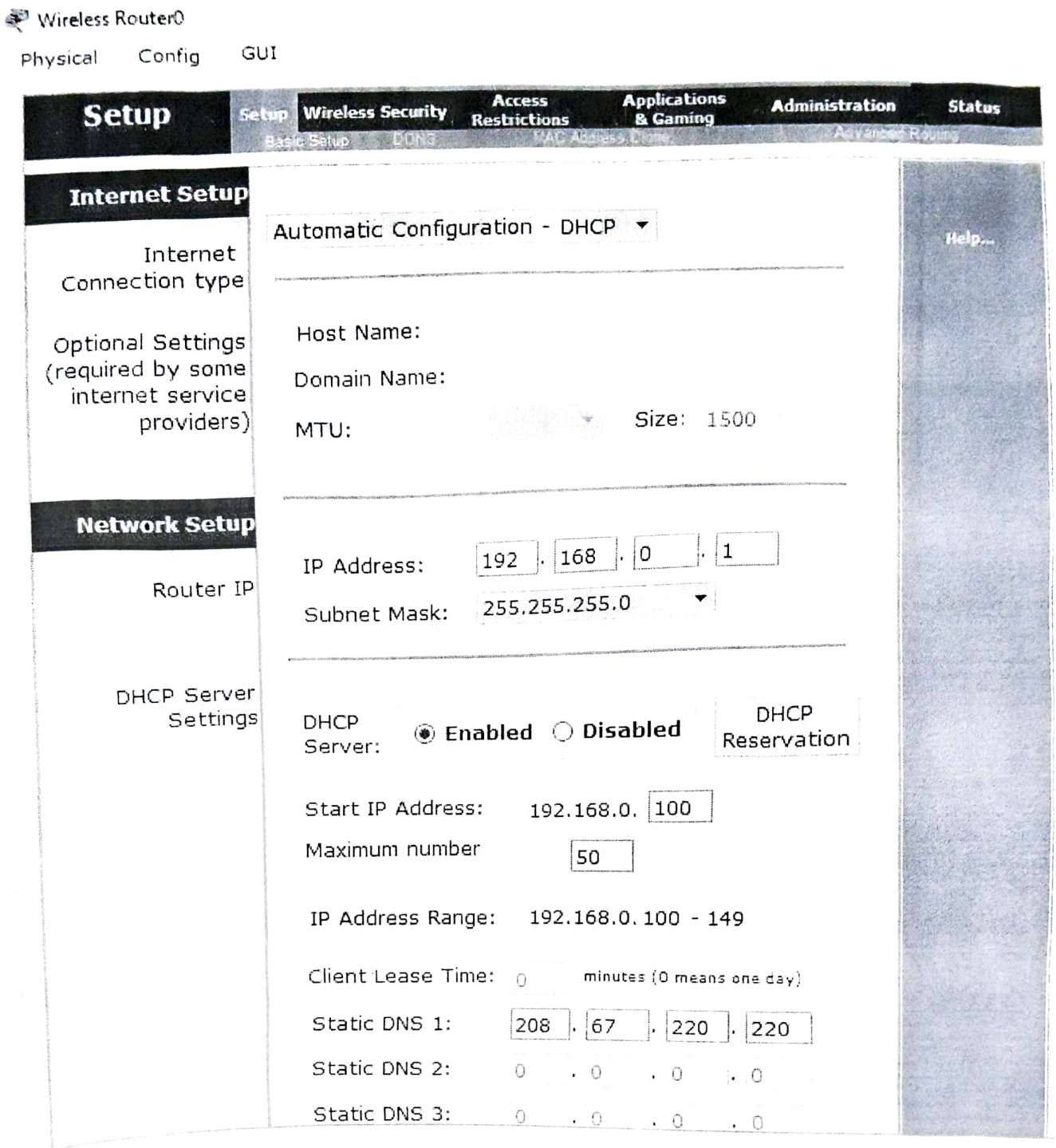

Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
192.168.10.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.10.1
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::201:42FF:FE97:C1DE
IPv6 Gateway
IPv6 DNS Server


PC1
Physical
Config
Desktop
Custom Interface
IP Configuration
IP Configuration
DHCP
Static
IP Address
192.168.20.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.20.1
DNS Server
0.0.0.0
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::203:E4FF:FE28:7496
IPv6 Gateway
IPv6 DNS Server


PC0
Physical
Config
Desktop
Custom Interface
Command Prompt
Packet Tracer PC Command Line 1.0
PC>ping 192.168.20.2
Pinging 192.168.20.2 with 32 bytes of data:
Request timed out.
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Ping statistics for 192.168.20.2:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
PC>


Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Set Tiled Background
Simulation Panel
Event List
Simulation
Time: 00:20:18.960
Power Cycle Devices
PLAY CONTROLS:
Back
Auto Capture / Play
Capture / Forward
Connections
Automatically Choose Connection Type
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Successful
PC0
PC1
ICMP
Cisco Packet Tracer Student
Logical
[Root]
New Cluster
Set Tiled Background
Viewport
Laptop-PT
Laptop0
PC-PT
PC0
WRT300N
Wireless Router0
Cable Modem0
Cloud-PT
Cloud0
Server-PT
Server0
Realtime
Time: 00:19:55
Power Cycle Devices
Fast Forward Time
Connections
Copper Straight-Through
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)


Wireless Router0
Physical
Config
GUI
Wireless-N
Broadband Router
Firmware Version: v0.93.3
Wireless-N Broadband Router
WRT300N
Wireless
Setup
Wireless
Security
Access Restrictions
Applications & Gaming
Administration
Status
Basic Wireless Settings
Wireless Security
Wireless MAC Filter
Advanced Wireless Settings
Basic Wireless Settings
Help...
Network Mode:
Mixed
Network Name (SSID):
HomeNetwork
Radio Band:
Auto
Wide Channel:
Auto
Standard Channel:
1 - 2.412GHz
SSID Broadcast:
Enabled
Disabled
Save Settings
Cancel Changes


Wireless Router0
Physical
Config
GUI
Setup
Setup
Wireless Security
Access Restrictions
Applications & Gaming
Administration
Status
Basic Setup
DDNS
MAC Address Clone
Advanced Routing
Internet Setup
Internet Connection type
Automatic Configuration - DHCP
Help...
Optional Settings (required by some internet service providers)
Host Name:
Domain Name:
MTU:
Size: 1500
Network Setup
Router IP
IP Address: 192 . 168 . 0 . 1
Subnet Mask: 255.255.255.0
DHCP Server Settings
DHCP Server: Enabled Disabled
DHCP Reservation
Start IP Address: 192.168.0. 100
Maximum number 50
IP Address Range: 192.168.0. 100 - 149
Client Lease Time: 0 minutes (0 means one day)
Static DNS 1: 208 . 67 . 220 . 220
Static DNS 2: 0 . 0 . 0 . 0
Static DNS 3: 0 . 0 . 0 . 0

EXP. NO: 10b

DATE:

WIRELESS INTERNETWORKING

AIM:

To design and configure an internetwork using a wireless router, DHCP server and Internet cloud.

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Add a wireless router, DHCP server and Internet cloud.
3. Add the required wireless devices.,
4. Connect the devices to the wireless router.
5. Configure the wireless router settings.
6. Configure the DHCP server with IP settings
7. Enable DHCP for automatic IP assignment.
8. Connect the wireless devices to the network.
9. Verify the assigned IP address.
10. Test connectivity between the devices and Internet cloud.

Result

Thus, the wireless internetwork was successfully configured and connectivity was verified.

EXP. NO: 11 a

DATE:

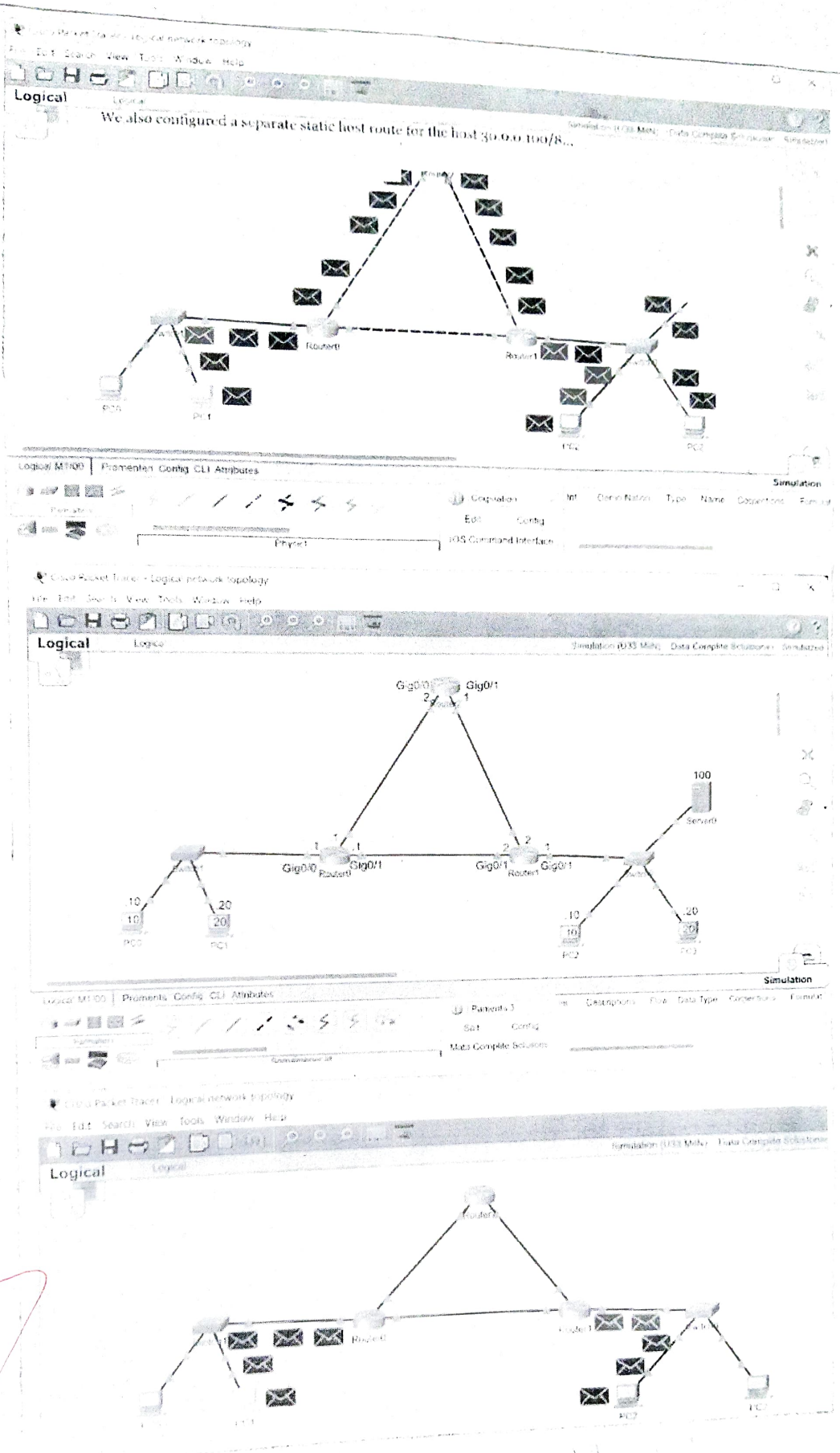
STATIC ROUTING

AIM:

To configure static routing using Cisco Packet Tracer.

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Create the required router and Pc topology.
3. Connect the devices using suitable cables.
4. Assign IP addresses to PCs and router interfaces.
5. Enable router interfaces using no shutdown.
6. Configure static routes using IP route.
7. Configure the required backup route.
8. Check the routing table using show IP route.
9. Test connectivity using ping.
10. Verify communication between the networks.



RESULT:

Thus, the echo client - server was successfully implemented using sockets.

EXP. NO : 11 b

DATE:

RIP ROUTING

AIM:

To configure RIP routing using Cisco Packet Tracer

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Create the required router and PC Topology.
3. Connect the devices using suitable cables.
4. Assign IP addresses to PCs and router interfaces.
5. Enable router interfaces using no shutdown.
6. Enter RIP configuration mode using router rip.
7. Add the connected networks using network commands
8. Configure RIP on the required routers.
9. Check the routing table.
10. Test connectivity using ping.

RESULT:

Thus, RIP routing was successfully configured and connectivity between the networks was verified.

EXP. NO : 12 a)

DATE:

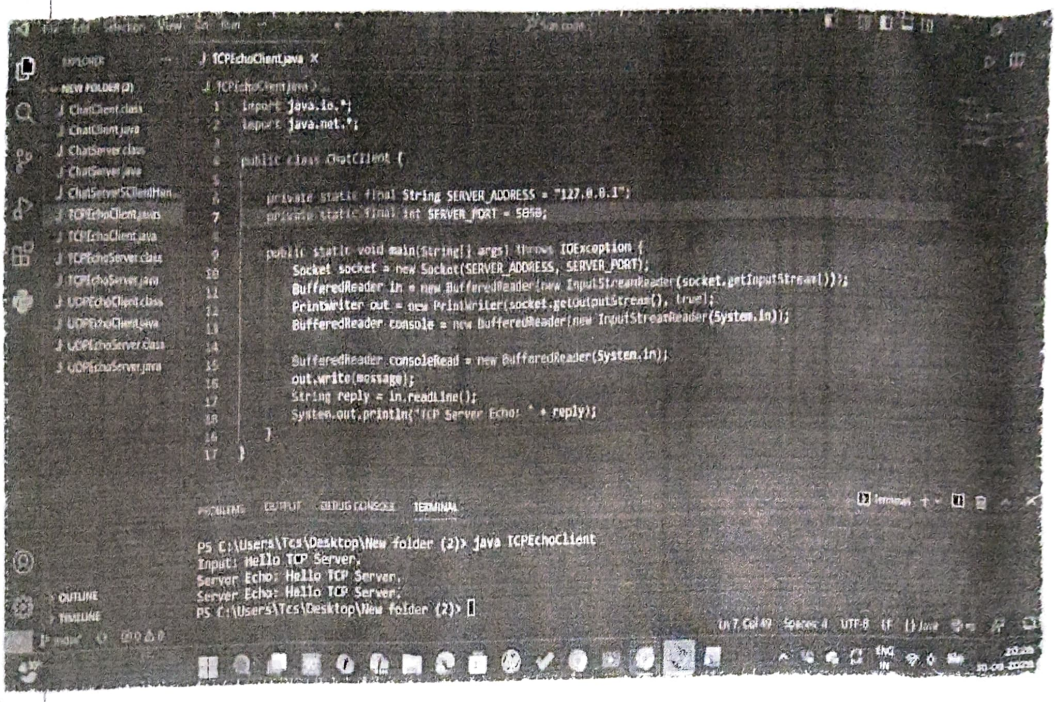
ECHO CLIENT - SERVER

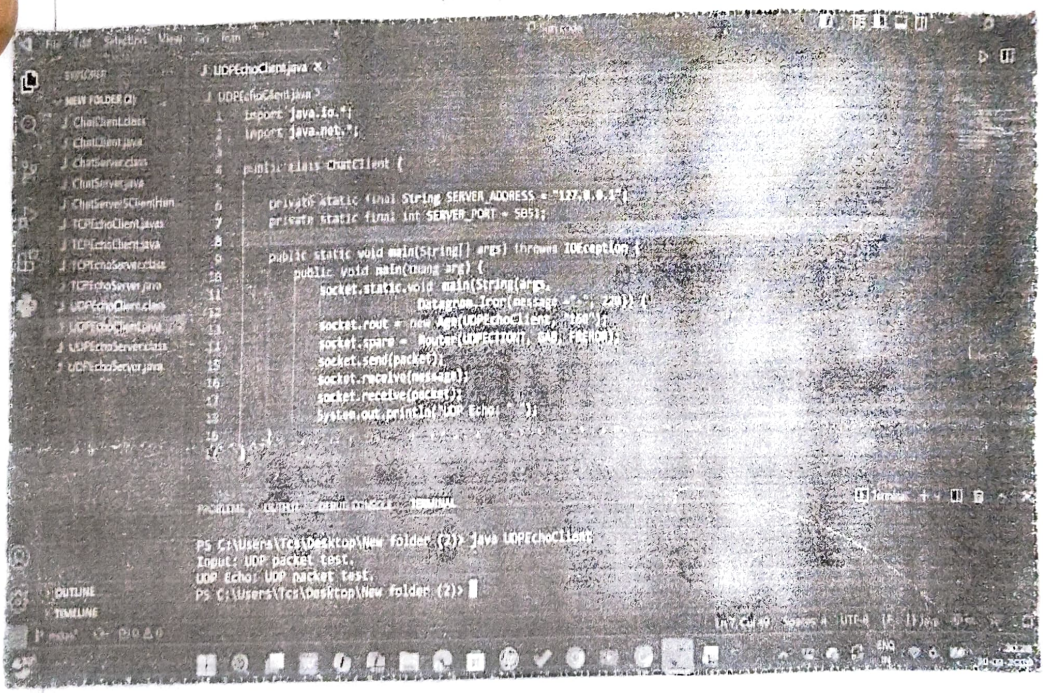
AIM:

To implement an echo client - server using TCP/UDP sockets.

ALGORITHM:

* Create the server socket.
* Bind the socket to an IP address and port.
* Start listening for client requests.
* Create the client socket
* Connect the client to the server.
* Enter a message from the client.
* Send the message to the server.
* Server receives and echoes the message.
* Client receives the echoed message.
* Close the sockets.





RESULT:

Thus, the echo client-server was successfully implemented using sockets.

EXP. NO : 12b)

DATE:

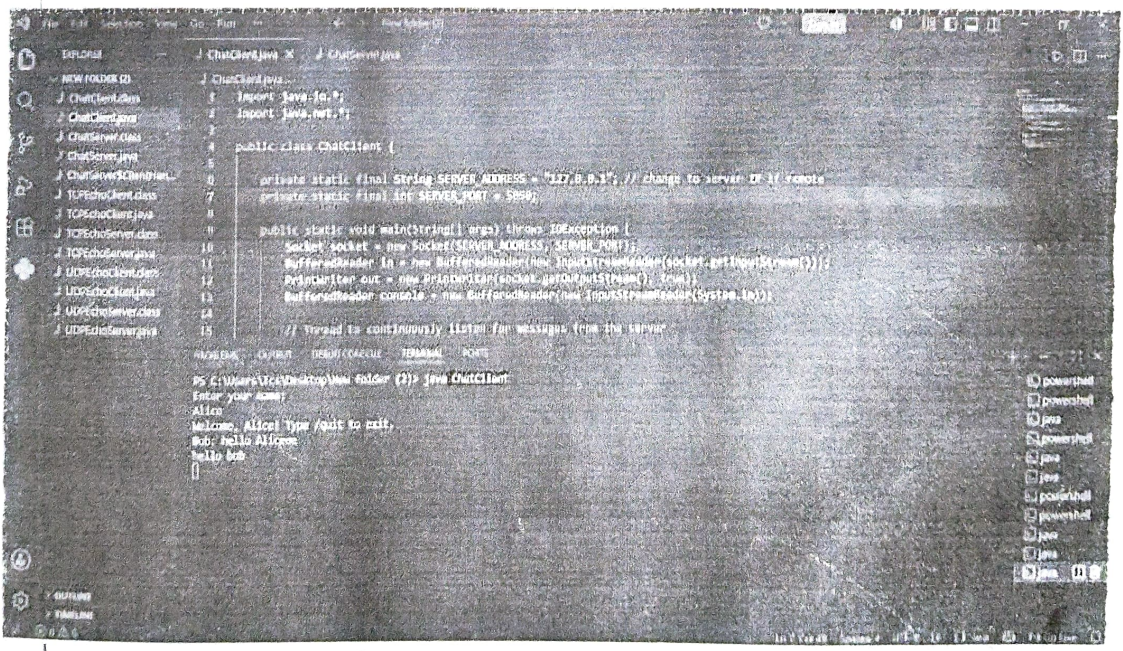
CHAT - CLIENT SERVER

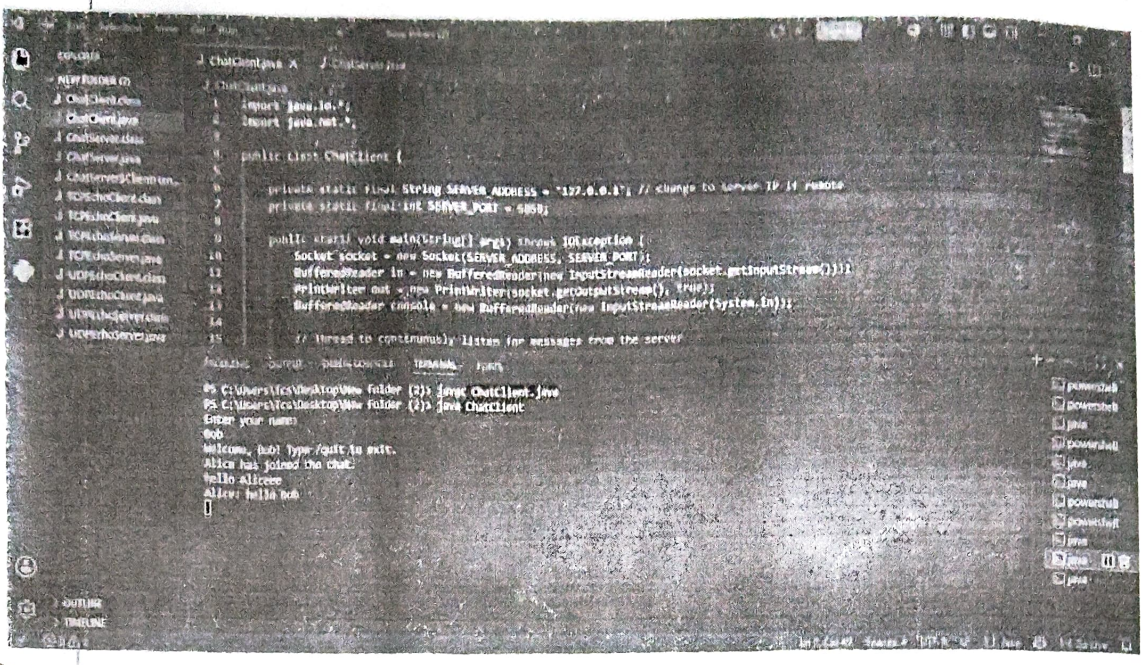
AIM:

To implement a chat program using socket programming

ALGORITHM:

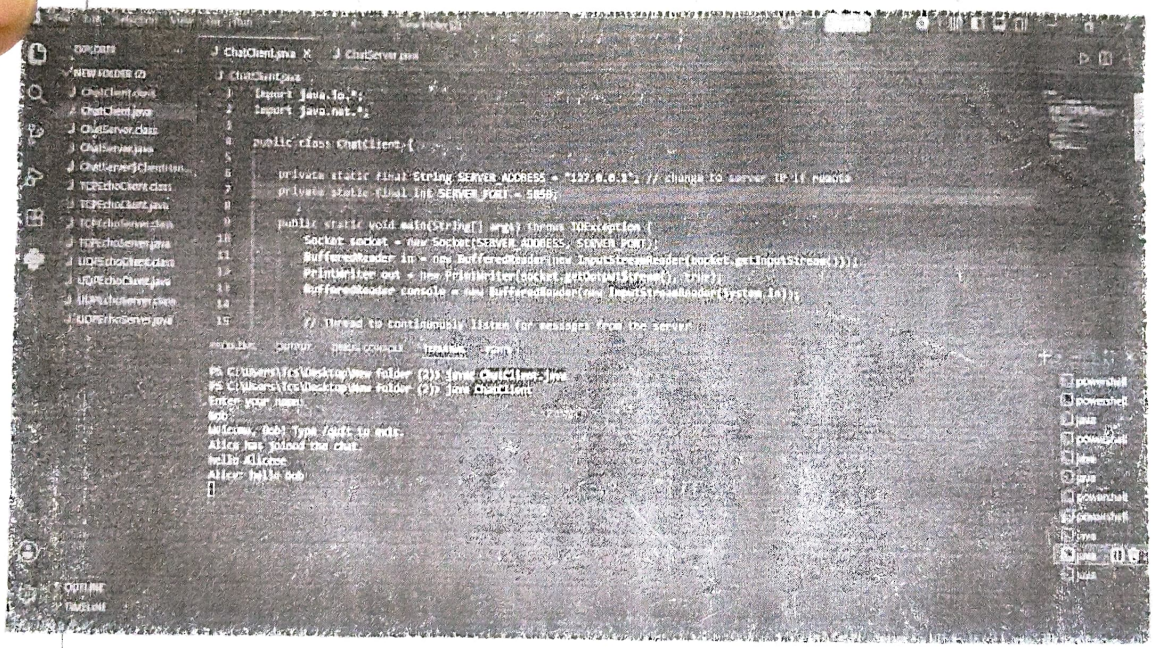
1. Create the server socket
2. Bind the socket to an IP address and port.
3. Start listening for a client.
4. Create the client socket.
5. Establish connection between client and server.
6. Enter a message from the client.
7. Send and receive messages between client and server.
8. Display the received messages.
9. Continue communication until termination.
10. Close the sockets.





```
import java.io.*;
import java.net.*;

public class ChatClient {

    private static final String SERVER_ADDRESS = "127.0.0.1"; // change to server IP if remote
    private static final int SERVER_PORT = 5050;

    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader console = new BufferedReader(new InputStreamReader(System.in));

        // Thread to continuously listen for messages from the server
```

```
PS C:\Users\fcs\Desktop\New folder (2)> javac ChatClient.java
PS C:\Users\fcs\Desktop\New folder (2)> java ChatClient
Enter your name:
Bob
Welcome, Bob! Type /quit to exit.
Alice has joined the chat.
Hello Alicee
Alice: hello Bob
```



```
import java.io.*;
import java.net.*;

public class ChatClient {

    private static final String SERVER_ADDRESS = "127.0.0.1"; // change to server IP if remote
    private static final int SERVER_PORT = 5050;

    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader console = new BufferedReader(new InputStreamReader(System.in));

        // Thread to continuously listen for messages from the server
```

```
PS C:\Users\fcs\Desktop\New folder (2)> javac ChatClient.java
PS C:\Users\fcs\Desktop\New folder (2)> java ChatClient
Enter your name:
Bob
Welcome, Bob! Type /quit to exit.
Alice has joined the chat.
Hello Alicee
Alice: hello Bob
```

RESULT:

Thus the chat client - server was successfully implemented using socket programming.

EXP. NO: 18
DATE:

IMPLEMENT YOUR OWN PING PROGRAM

AIM:

To implement a Ping program using Python raw sockets to check the reachability of a host and calculate the round trip time.

CODE:

```
import socket
import struct
import time
import os
def checksum (data):
    total = 0
    for i in range (0, len(data) -1, 2):
        total += (data [i] << 8) + data [i+1]
    if len (data) % 2 :
        total += data [-1] << 8
    total = (total >> 16) + (total & 0xffff)
    total += total >> 16
    Return (~total) & 0xffff
def ping (host):
    try :
        dest_addr = socket.gethostbyname (host)
        sock = socket. socket (socket. AF_INET,
                               Socket. SOCK_RAW,
                               Socket . IPPROTO - ICMP)
        packet_id = os. getpid() & 0xffff
        Sequence = 1
        header = struct. pack (
                 "!BBHHH",
                  8,
                  0,
                  0,  packet_id, sequence)
```

```
        data = b"PING"
        chksum = checksum(header + data)
        header = struct.pack(
            "!BBHHH",
            8,
            0
            chksum,
            packet_id,
            sequence
        )
        packet = header + data
        start = time.time()
        sock.sendto(packet, dest_addr, 1))
        sock.settimeout(2)
        sock.recvfrom(1024)
        end = time.time()
        rtt = (end - start) * 1000
        print("Reply from", dest_addr,
                    ": time = %.2f ms" % rtt)

        sock.close()
    except socket.timeout:
        print("Request timed out")
except PermissionError:
        print("Permission denied. Run the
    program as administrator (root.")
host = input("Enter destination host: ")
ping(host)
```

SAMPLE INPUT / OUTPUT:

```
Enter destination host: 8.8.8.8
Reply from 8.8.8.8 : time = 24.53 ms
```

RESULT:

Thus, the ping program was successfully implemented in Python using raw sockets.

EXP.NO: 14
DATE:

PACKET SNIFFING USING RAW SOCKETS

AIM:

To implement a packet sniffing using Python raw sockets and display the source IP, destination IP and protocol of captured packets.

CODE:

```
import socket
import struct
sock = socket.socket(
        socket.AF_INET,
        socket.SOCK_RAW,
        socket.IPPROTO_IP
    )
print("Packet Sniffer Started...")
print("Waiting for packets...\n")
while True:
    packet, address = sock.recvfrom(65535)
    ip_header = packet[0:20]
    iph = struct.unpack(
            "!BBHHHBBH4s4s",
            ip_header
        )
    source_ip = socket.inet_ntoa(iph[8])
    destination_ip = socket.inet_ntoa(iph[9])
    protocol = iph[6]
    print("-----------------")
    print("Source IP :", source_ip)
    print("Destination IP :", destination_ip)
    print("Protocol :", protocol)
    print("Packet Size :", len(packet), "bytes")
    print("-----------------")
```

SAMPLE OUTPUT

```
Packet Sniffer started...
Waiting for packets....
----------------------------
Source IP      : 192.168.1.10
Destination IP : 142.250.183.14
Protocol       : 6
Packet size    : 60 bytes
----------------------------
----------------------------
Source IP      : 8.8.8.8
Destination IP : 192.168.1.10
Protocol       : 1
Packet size    : 84 bytes
----------------------------
```

RESULT:

Thus, the Ping Program and packet sniffing program were successfully implemented using Python raw sockets.

EXP. NO: 15
DATE:

ANALYSIS OF VARIOUS TYPES OF SERVERS USING WEBALIZER

AIM:

To analyze the web server log files using the Webalizer tool and study various server statistics such as number of requests, visitors, transfered data, URLs and response status codes.

ALGORITHM:

1. Start the Webalizer application.
2. Install and configure Webalizer on the system.
3. Locate the web server access log file.
4. Configure Webalizer with the required log-file path and output directory.
5. Run Webalizer on the selected log file.
6. Webalizer reads and analyzes the server log entries.
7. It generates statistics such as:
   - Number of visits
   - Number of requests
   - Pages accessed
   - Data transferred
   - HTTP response code
   - Referring sites
   - Operating systems and browsers.
8. Open the generated HTML report in a web browser.
9. Analyze the server activity using the generated graphs and tables.
10. Repeat the analysis for different server log files if required.

INPUT:

Web server access log files containing information such as:

Client IP
Date and Time
Requested URL
HTTP Method
HTTP status code
Bytes Transferred
Referrer
User Agent

Example input:

access. log.

OUTPUT:

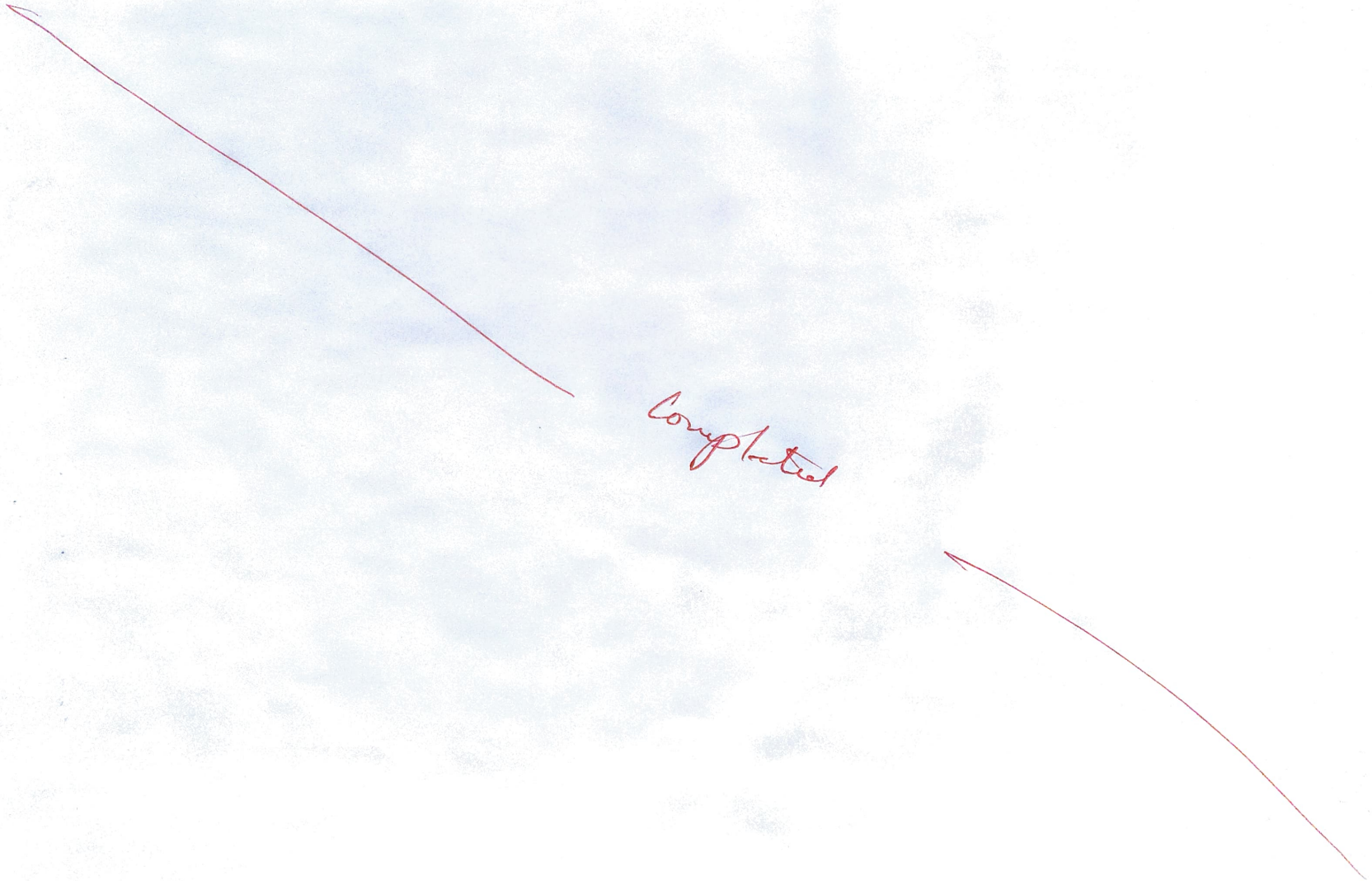
Webalizer generates an HTML - based Statistical report containing

Webalizer Statistics

Total Requests : 1250
Total visits : 480
Total Pages : 920
Total Bytes : 15.4 mb
Successful Requests : 1180
404 Bytes Errors : 45

RESULT:

Thus, the Webalizer tool was used successfully to analyze web server log files, and various server statistics such as Requests, visitors, pages, bandwidth usage and HTTP responses code were obtain


Completed